

Apprentissage par renforcement

Enzo Claeysen, Smac Cristal

Sommaire :

1. Qu'est-ce que l'apprentissage par renforcement ?

2. Comment créer son environnement ?

3. Utilisation des librairies de RL

4. Paramétrage et environnement

5. Conclusion et ouvertures

1/Introduction

À travers un petit exemple :

- Principe de l'apprentissage par renforcement (RL) ?
- Vocabulaire important ?
- Quand faut-il utiliser le RL ?

Petit exemple

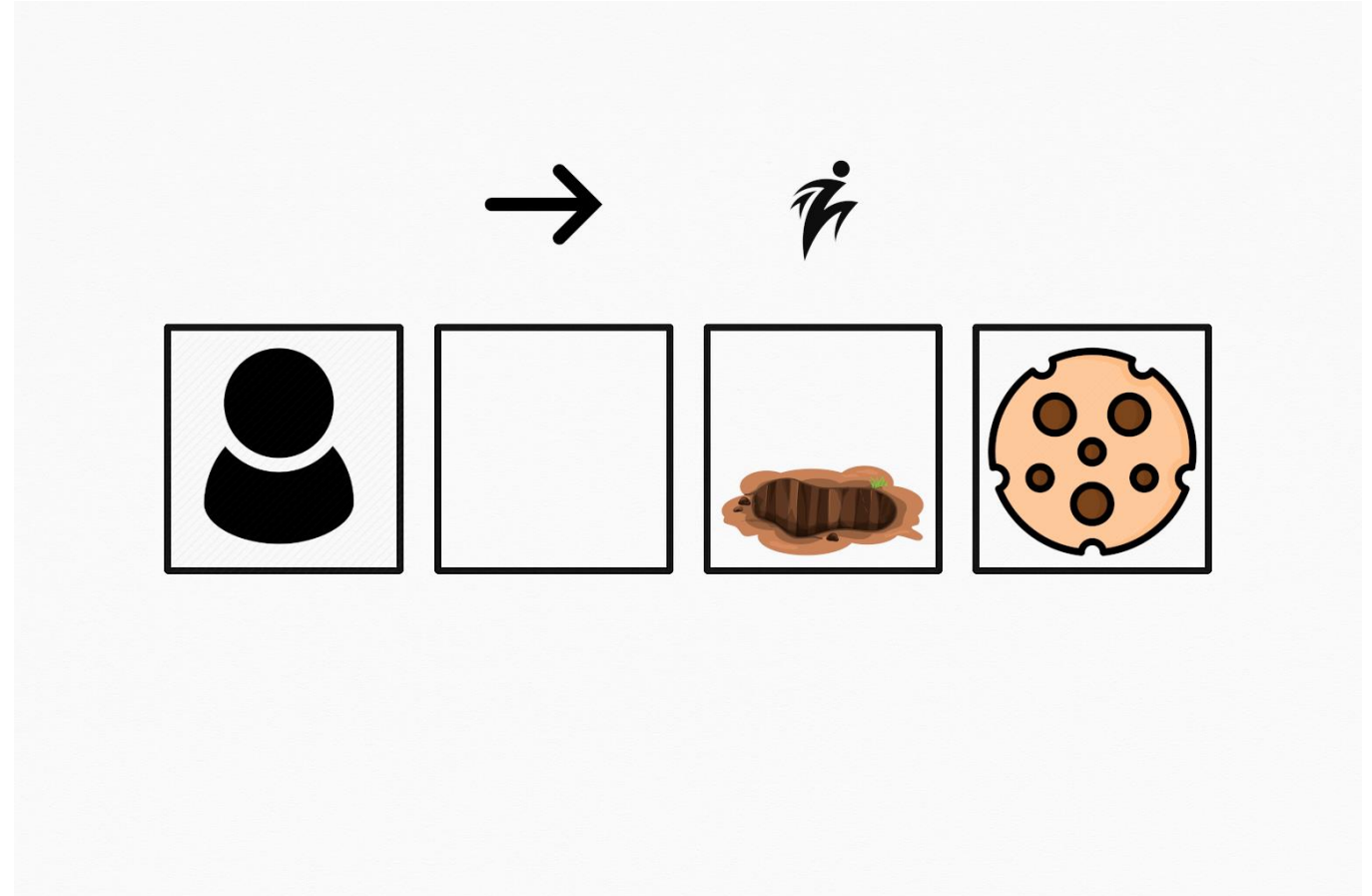
- 4 cases
- 1 cookie
- 1 trou
- 1 bonhomme

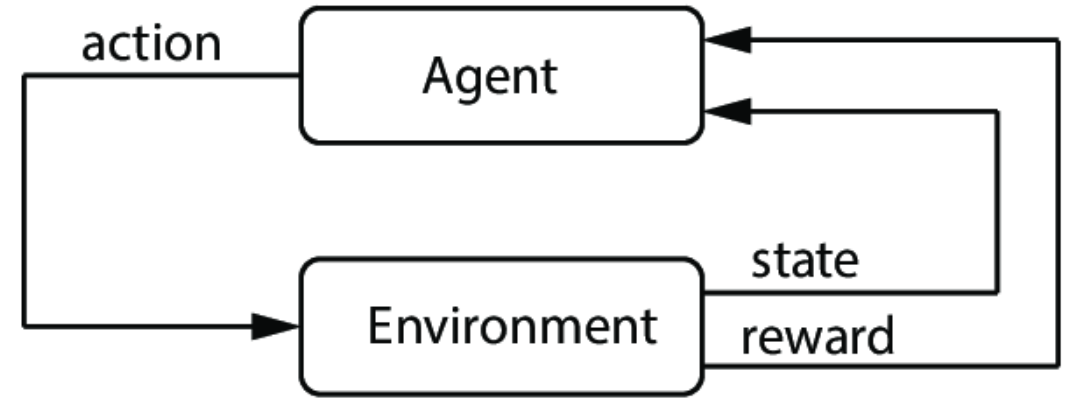
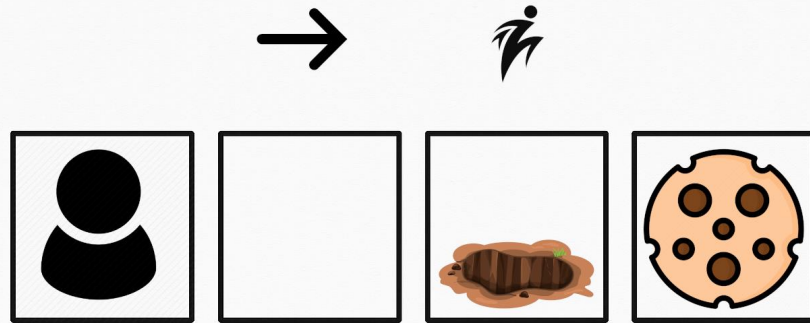


Petit exemple

L'environnement

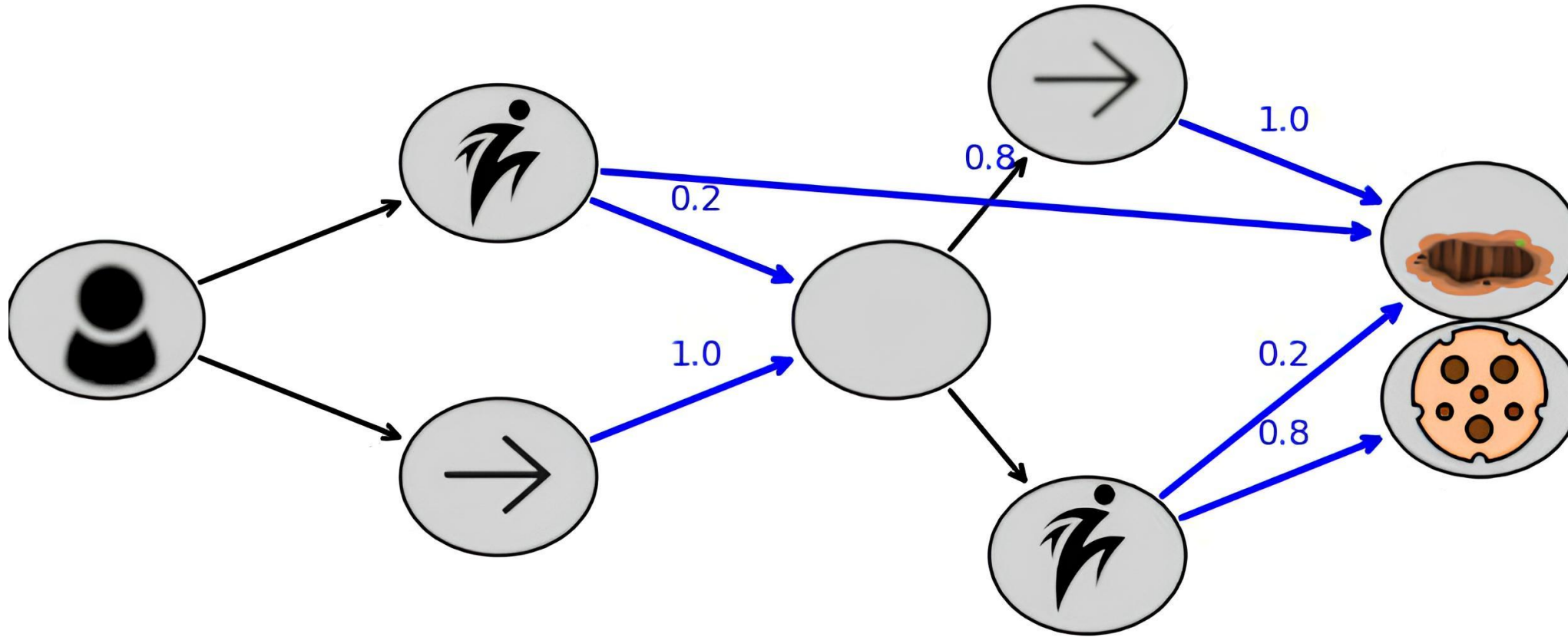
- 4 états
- 2 actions





Intéraction Agent- Environnement

- **Step** : Une interaction agent - environnement
- **Épisode** : Une succession de steps allant d'un état initial à un état final
- **Récompense** : Scalaire obtenu par l'agent, détermine si l'état actuel est favorable ou non.



Représentation par graphe

Objectif : trouver le comportement qui maximise le gain espéré

Quand pouvons-nous utiliser le RL ?

En théorie : lorsque l'on peut représenter son problème par le graphe précédent.

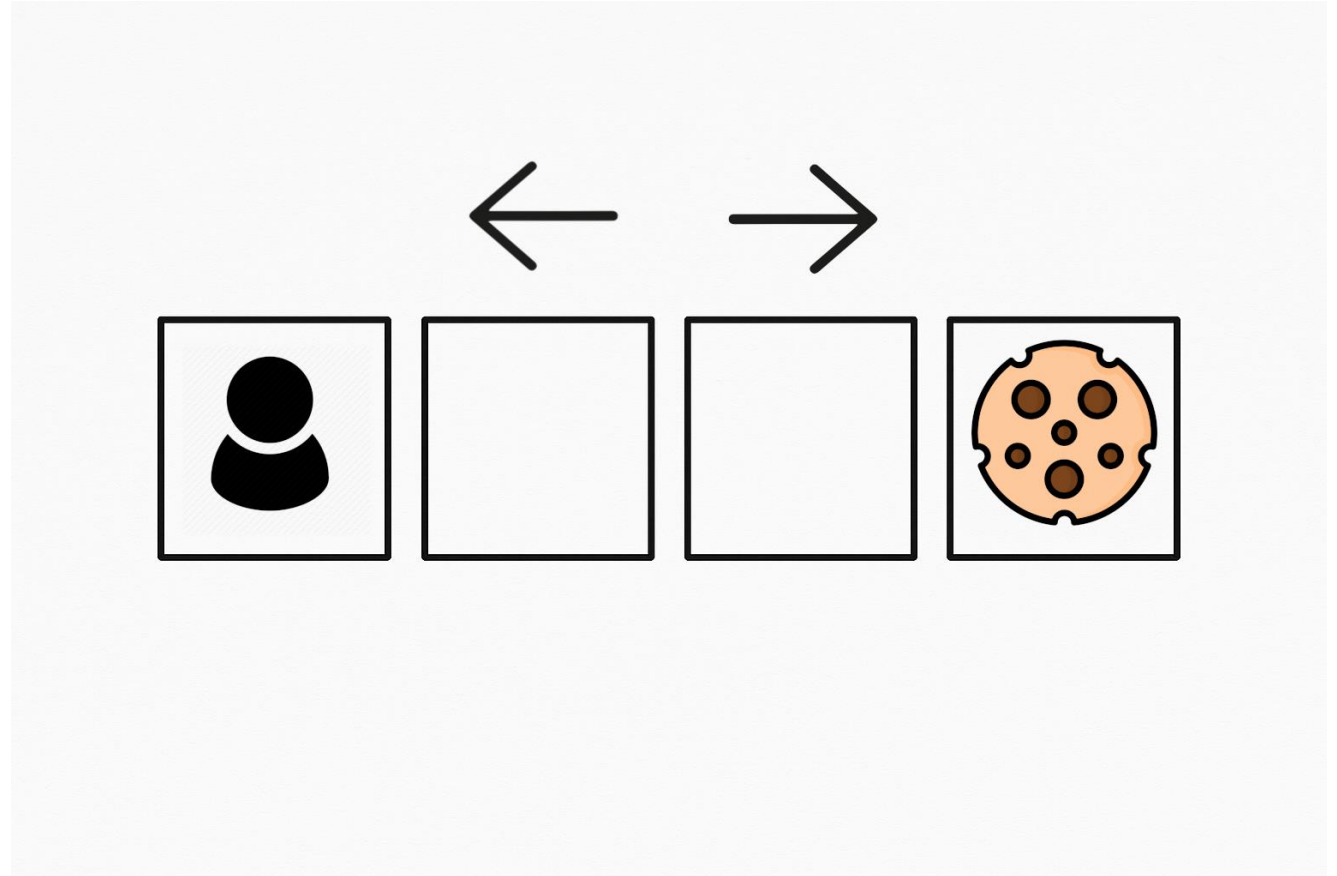
Mais en pratique, ...

2/Comment créer son environnement ?

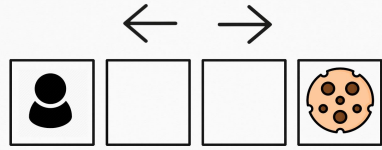
- Comment créer un environnement à partir d'un problème ?
- Nouvel exemple : le couloir

Le Couloir

- État initial :
case de gauche
- État final :
Cookie (favorable)
- 4 états :
Position de l'agent
- 2 actions :
Gauche
Droite



Le Couloir



1. Créer une classe

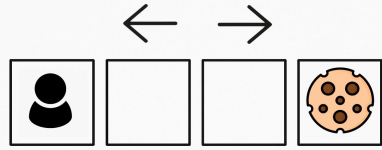
On utilise `gymnasium`.

Doit hériter de `gymnasium.Env`

```
import gymnasium
```

```
class Couloir(gymnasium.Env) :
```

Le Couloir



1. Créer une classe
2. Créer le constructeur

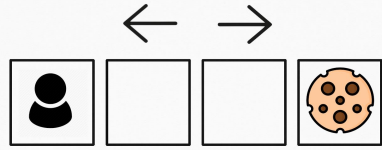
Méthode `__init__(self, options)`

```
import gymnasium
```

```
class Couloir(gymnasium.Env) :
```

```
    def __init__(self, options = None) :
```

Le Couloir



1. Créer une classe
2. Créer le constructeur

Méthode `__init__(self, options)`

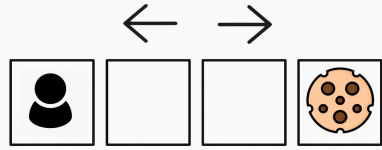
Appel au constructeur de la super classe

```
import gymnasium
```

```
class Couloir(gymnasium.Env) :
```

```
    def __init__(self, options = None) :  
        super(Couloir, self).__init__()
```

Le Couloir



1. Créer une classe

2. Créer le constructeur

Méthode `__init__(self, options)`

Appel au constructeur de la super classe

`action_space`

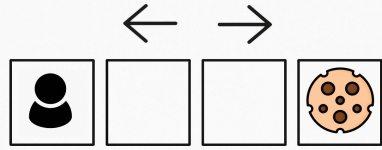
```
import gymnasium
from gymnasium import spaces
```

```
class Couloir(gymnasium.Env) :
```

```
    def __init__(self, options = None) :
        super(Couloir, self).__init__()
```

```
        self.action_space = spaces.Discrete(2)
```

Le Couloir



1. Créer une classe

2. Créer le constructeur

Méthode `__init__(self, options)`

Appel au constructeur de la super classe

`action_space`

`observation_space`

```
import gymnasium
from gymnasium import spaces
```

```
class Couloir(gymnasium.Env) :
```

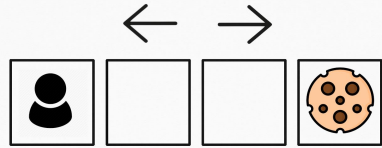
```
    def __init__(self, options = None) :
        super(Couloir, self).__init__()
```

```
        self.action_space = spaces.Discrete(2)
```

```
        self.size = options['n_states']
        self.observation_space = spaces.Discrete(self.size)
```

```
# Créer un couloir de taille 3
Couloir(
    options = {
        'n_states' : 3
    }
)
```

Le Couloir



1. Créer une classe

2. Créer le constructeur

Méthode `__init__(self, options)`

Appel au constructeur de la super classe

`action_space`

`observation_space`

Initialisation de l'environnement

```
import gymnasium
from gymnasium import spaces
```

```
class Couloir(gymnasium.Env) :
```

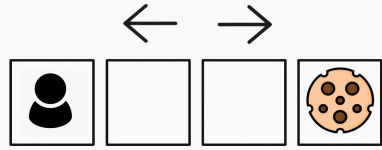
```
    def __init__(self, options = None) :
        super(Couloir, self).__init__()
```

```
        self.action_space = spaces.Discrete(2)
```

```
        self.size = options['n_states']
        self.observation_space = spaces.Discrete(self.size)
```

```
        self.position = 0
```

Le Couloir

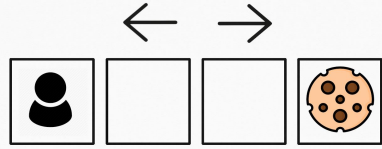


```
def reset(self, seed = None, options = None) :
```

1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`

Seed : utile si environnement stochastique

Le Couloir



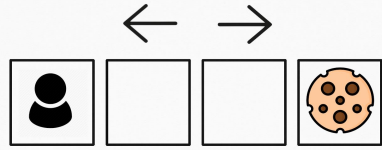
```
def reset(self, seed = None, options = None) :  
    super().reset(seed = seed)
```

1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`

Seed : utile si environnement stochastique

Appel à la super classe

Le Couloir



1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`

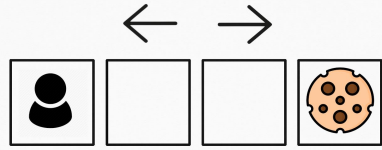
Seed : utile si environnement stochastique

Appel à la super classe

Réinitialisation de l'environnement

```
def reset(self, seed = None, options = None) :  
    super().reset(seed = seed)  
  
    self.position = 0
```

Le Couloir



1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`

Seed : utile si environnement stochastique

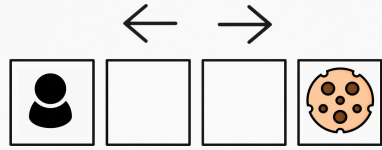
Appel à la super classe

Réinitialisation de l'environnement

Renvoie état initial + infos

```
def reset(self, seed = None, options = None) :  
    super().reset(seed = seed)  
  
    self.position = 0  
  
    return self.position, {}
```

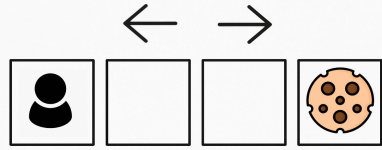
Le Couloir



```
def step(self, action) :
```

1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`
4. Créer la méthode `step()`

Le Couloir



1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`
4. Créer la méthode `step()`

Appliquer l'action

```
def step(self, action) :
```

```
# On applique l'action
```

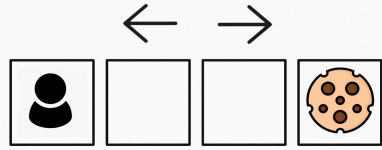
```
if action == 0 and self.position > 0 :
```

```
    self.position -= 1
```

```
elif action == 1 and self.position < self.size-1 :
```

```
    self.position += 1
```

Le Couloir



1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`
4. Créer la méthode `step()`

Appliquer l'action

Vérifier si un état final est atteint

```
def step(self, action) :
```

```
    # On applique l'action
```

```
    if action == 0 and self.position > 0 :
```

```
        self.position -= 1
```

```
    elif action == 1 and self.position < self.size-1 :
```

```
        self.position += 1
```

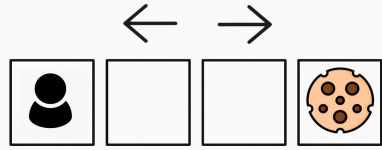
```
    # Vérifier si l'épisode est terminé
```

```
    done = False
```

```
    if self.position == self.size-1 :
```

```
        done = True
```

Le Couloir



1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`
4. Créer la méthode `step()`

Appliquer l'action

Vérifier si un état final est atteint

Calculer la récompense obtenue

```
def step(self, action) :
```

```
# On applique l'action
```

```
if action == 0 and self.position > 0 :
```

```
    self.position -= 1
```

```
elif action == 1 and self.position < self.size-1 :
```

```
    self.position += 1
```

```
# Vérifier si l'épisode est terminé
```

```
done = False
```

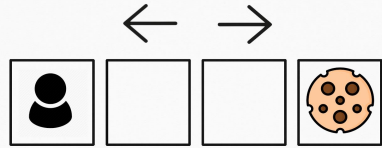
```
reward = -1
```

```
if self.position == self.size-1 :
```

```
    reward = 10
```

```
done = True
```

Le Couloir



1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`
4. Créer la méthode `step()`

Appliquer l'action

Vérifier si un état final est atteint

Calculer la récompense obtenue

Renvoyer les résultats

```
def step(self, action) :
```

```
# On applique l'action
```

```
if action == 0 and self.position > 0 :  
    self.position -= 1
```

```
elif action == 1 and self.position < self.size-1 :  
    self.position += 1
```

```
# Vérifier si l'épisode est terminé
```

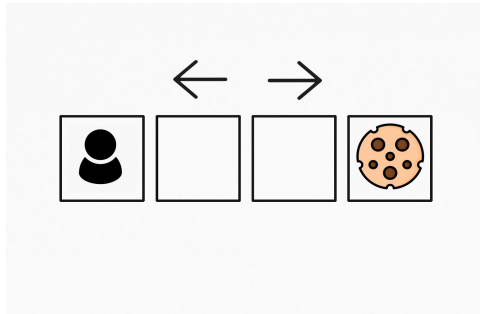
```
done = False
```

```
reward = -1
```

```
if self.position == self.size-1 :  
    reward = 10  
    done = True
```

```
return self.position, reward, done, False, {}
```

Le Couloir



1. Créer une classe
2. Créer le constructeur
3. Créer la méthode `reset()`
4. Créer la méthode `step()`

```
# Créer un couloir de taille 3
Couloir(
    options = {
        'n_states' : 3
    }
)
```

```
class Couloir(gymnasium.Env) :

    def __init__(self, options = None) :
        super(Couloir, self).__init__()

        self.action_space = spaces.Discrete(2)

        self.size = options['n_states']
        self.observation_space = spaces.Discrete(self.size)

        self.position = 0

    |

    def reset(self, seed = None, options = None) :
        super().reset(seed = seed)

        self.position = 0

        return self.position, {}

    def step(self, action) :

        # On applique l'action
        if action == 0 and self.position > 0 :
            self.position -= 1
        elif action == 1 and self.position < self.size-1 :
            self.position += 1

        # Vérifier si l'épisode est terminé
        done = False
        reward = -1
        if self.position == self.size-1 :
            reward = 10
            done = True

        return self.position, reward, done, False, {}
```

Et après ?

stable baselines permet de vérifier l'environnement.

```
from stable_baselines3.common.env_checker import check_env

# Créer un couloir de taille 3
env = Couloir(
    options = {
        'n_states' : 3
    }
)

check_env(env)
```

Si tout est correct, on peut commencer à entraîner notre agent !

3/Utilisation des librairies de RL

Dans cette partie nous verrons comment :

- Entraîner un agent avec notre implémentation du QLearning
- Entraîner un agent à l'aide de StableBaselines
- Entraîner un agent à l'aide de RLlib

QLearning

Importer notre implémentation
du QLearning

Créer un objet de notre
environnement

Utiliser `train_q_learning()`

**Attention entraînement en un
seul appel**

Afficher les résultats

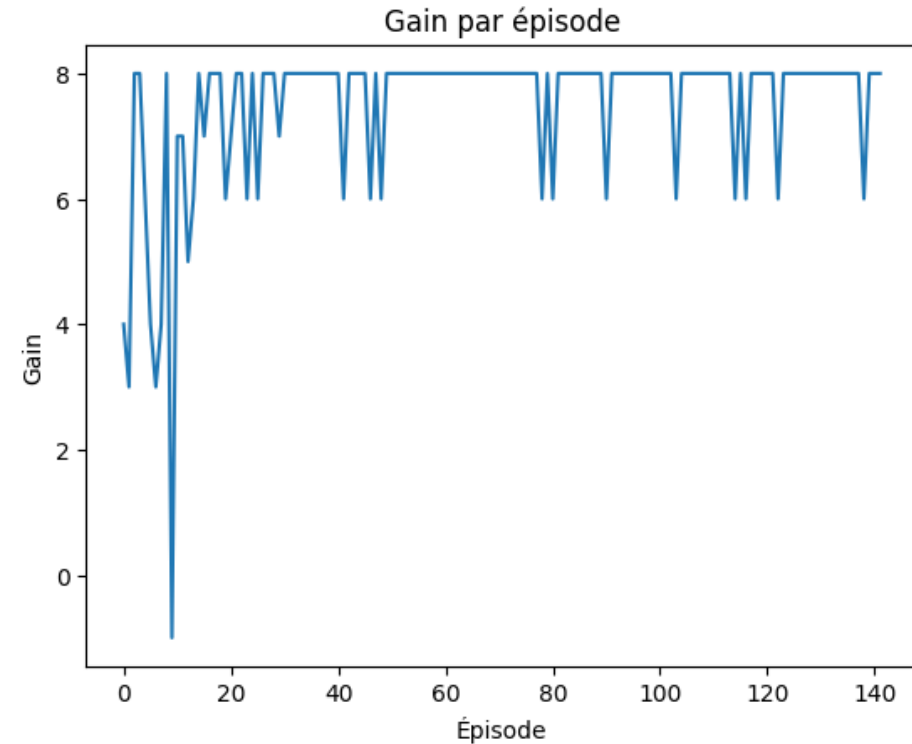
```
from QLearning import *

env = Couloir(options = {
    'n_states' : 4
})

model, gains, _ = train_q_learning(env, timesteps = 500)

plt.plot(gains)
plt.xlabel('Épisode')
plt.ylabel('Gain')
plt.title('Gain par épisode')

plt.show()
```



QLearning - Exploitation

On peut ajouter une
méthode `render()` pour
l'affichage

Sinon utiliser :

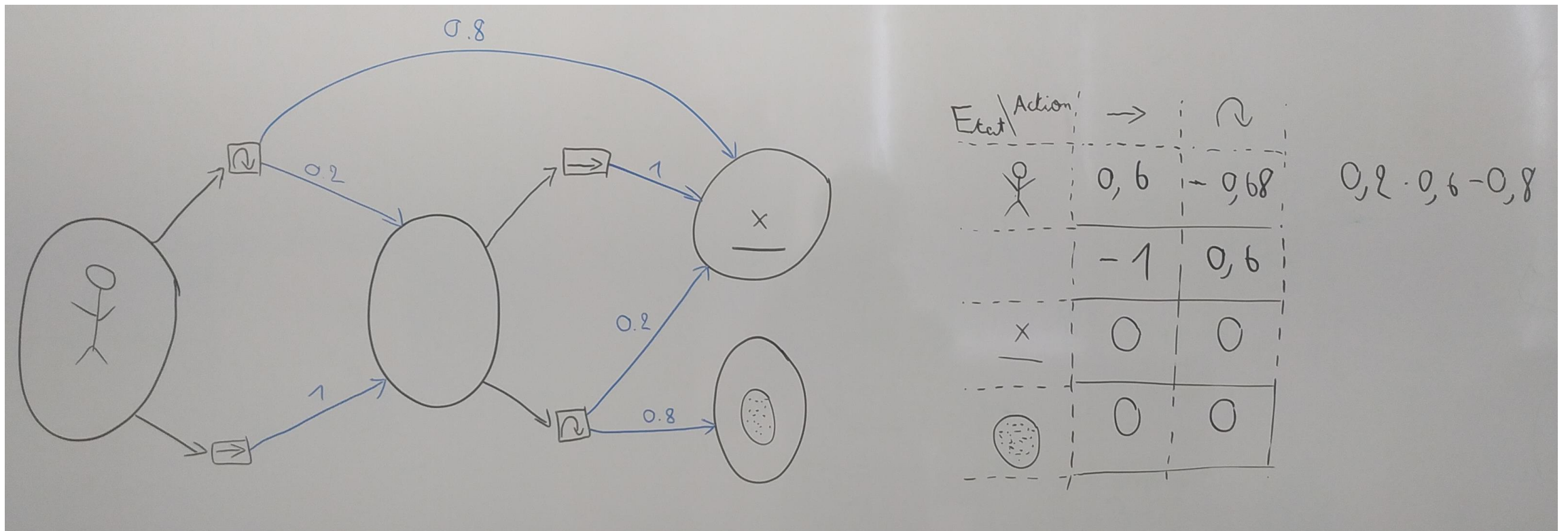
```
gain = test_q_learning(env, model, render = False, maxTimesteps = 10)
print(f"Gain obtenu : {gain}")
```

Gain obtenu : 1

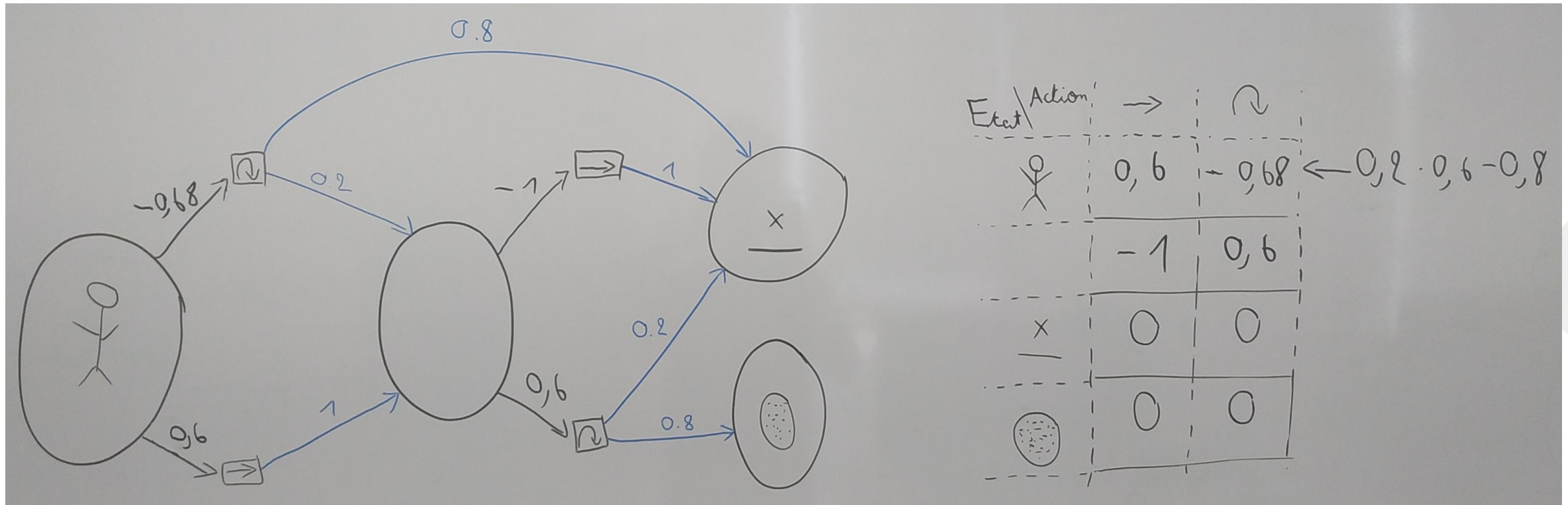
```
gain = test_q_learning(env, model)
print(f'Gain obtenu : {gain}')
```

```
+---++---++---++---+
| X ||   ||   ||   |
+---++---++---++---+
+---++---++---++---+
|   || X ||   ||   |
+---++---++---++---+
+---++---++---++---+
|   ||   || X ||   |
+---++---++---++---+
+---++---++---++---+
|   ||   ||   || X |
+---++---++---++---+
Gain obtenu : 1
```

QLearning - Fonctionnement

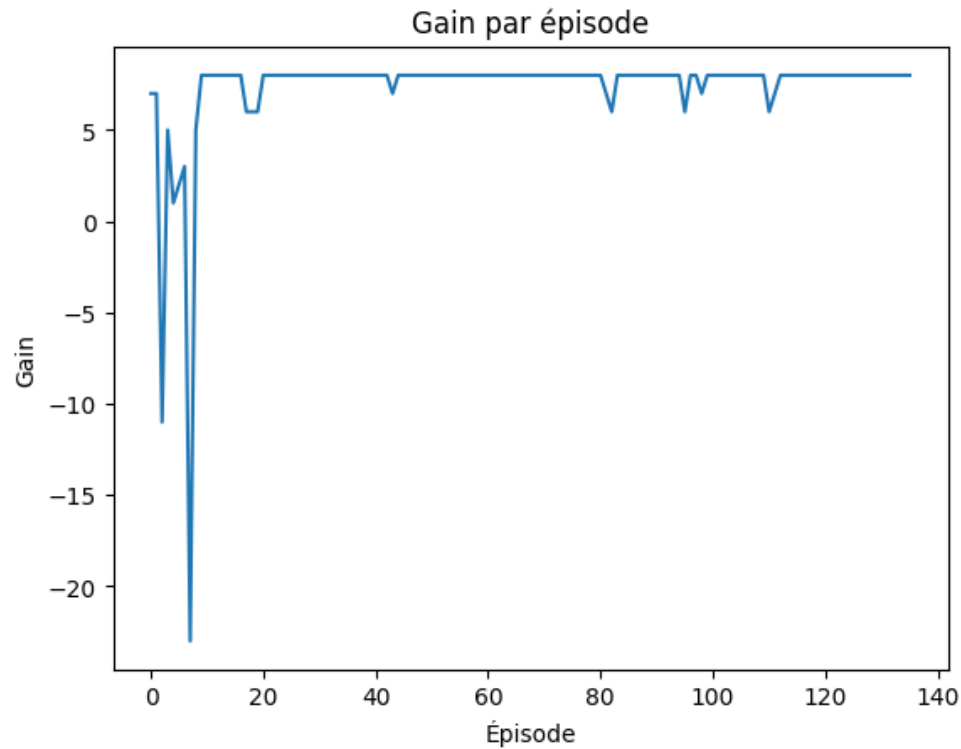


QLearning - Fonctionnement



DQN de StableBaselines

Entraînement



```
from stable_baselines3 import DQN

env = Couloir({
    "n_states" : 4
})

algo = DQN(
    "MlpPolicy",
    env
)

class RewardLogger(BaseCallback):
    def __init__(self):
        super().__init__()
        self.rewards = []
        self.current_rewards = 0

    def _on_step(self):
        self.current_rewards += self.locals["rewards"][0]
        if self.locals["dones"][0]:
            self.rewards.append(self.current_rewards)
            self.current_rewards = 0
        return True

logger = RewardLogger()

model = algo.learn(
    total_timesteps = 500,
    callback = logger
)

plt.plot(logger.rewards)
plt.xlabel('Épisode')
plt.ylabel('Gain')
plt.title('Gain par épisode')

plt.show()
```

DQN de StableBaselines

Exploitation

```
from stable_baselines3.common.env_util import make_vec_env
vec_env = make_vec_env(lambda : env, n_envs = 1)

obs = vec_env.reset()
done = False
gain = 0
vec_env.env_method('render')

while not done :
    action, _ = model.predict(obs, deterministic = True)

    obs, reward, done, info = vec_env.step(action)
    gain += reward

    vec_env.env_method('render')

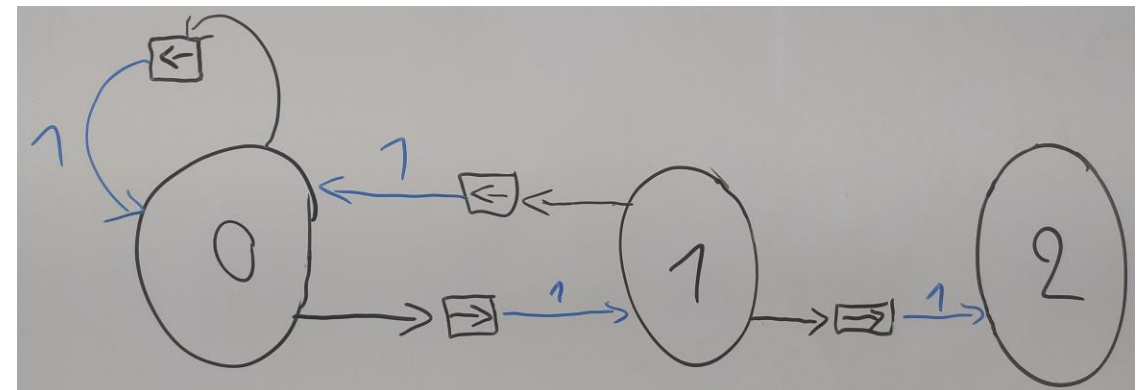
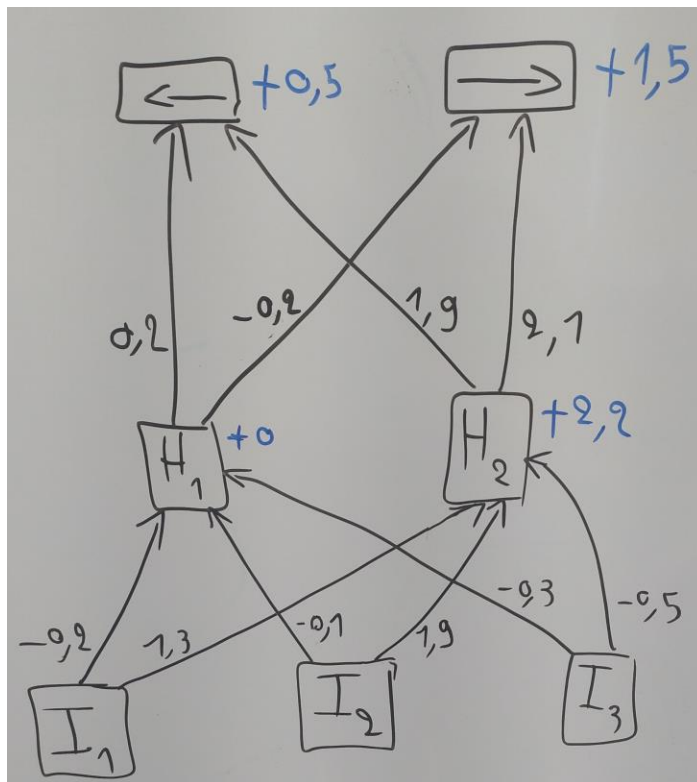
print(f"Gain obenu : {gain}")
```

```
+---++---++---++---+
| X ||  ||  ||  |
+---++---++---++---+
+---++---++---++---+
|  || X ||  ||  |
+---++---++---++---+
+---++---++---++---+
|  ||  || X ||  |
+---++---++---++---+
+---++---++---++---+
| X ||  ||  ||  |
+---++---++---++---+
Gain obenu : [8.]
```

DQN

StableBaselines

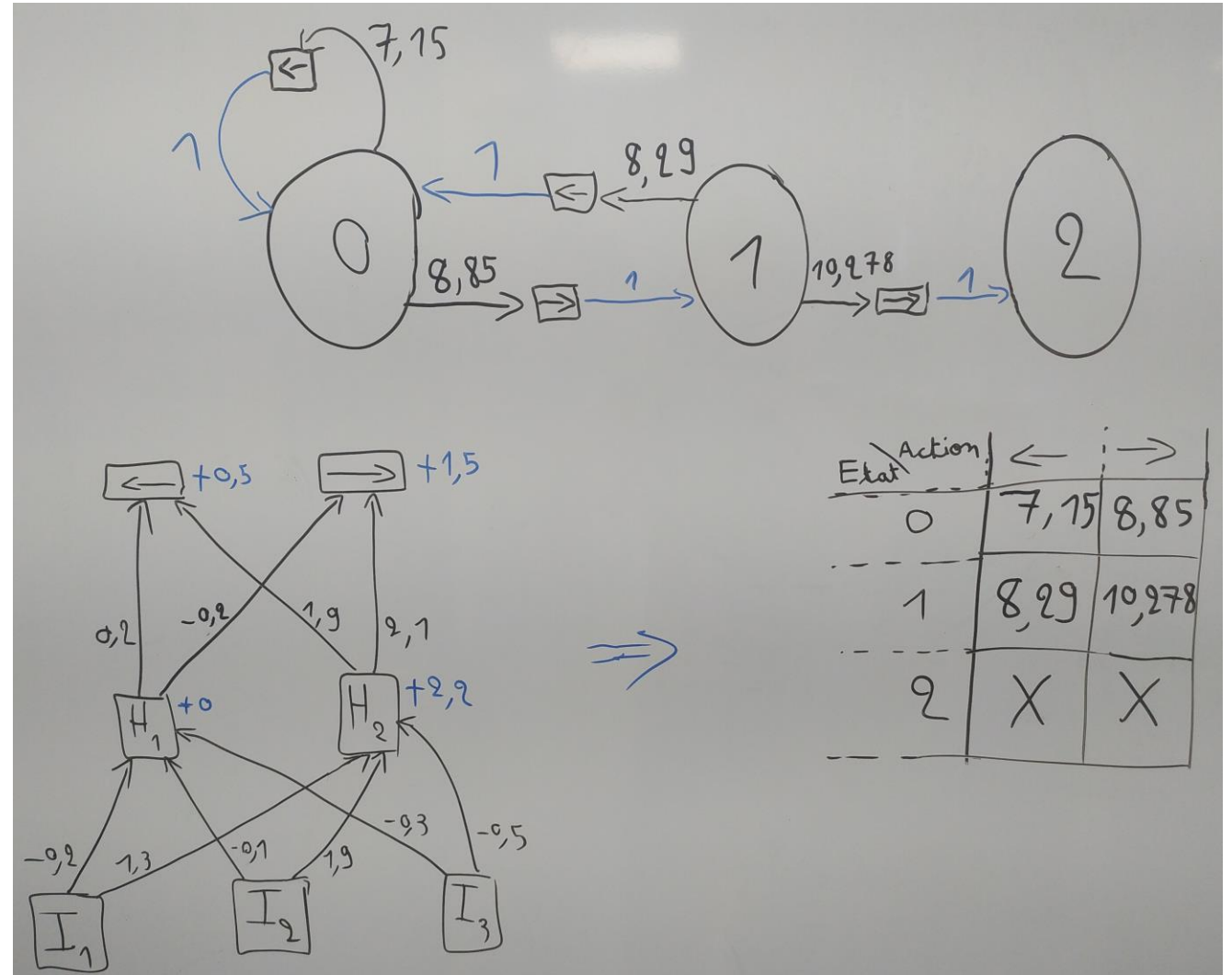
Fonctionnement



DQN

StableBaselines

Fonctionnement



DQN de RLlib

Entraînement

```
import ray
from ray import train
from ray.rllib.algorithms.dqn import DQNConfig
from ray.rllib.connectors.env_to_module import FlattenObservations

timesteps = 6000

config = (
    DQNConfig()
    .environment(
        Couloir,
        env_config = {'n_states' : 4}
    )
    .env_runners(
        env_to_module_connector = lambda env: FlattenObservations()
    )
)

algo = config.build_algo()

ordonnees = []
toPlot = []
episode_count = 0
timesteps_done = 0

while timesteps_done < timesteps :
    # On entraîne pour un certain nombre de steps
    results = algo.train()

    # On récupère le nombre de steps effectués en tout
    timesteps_done = results['env_runners']['num_env_steps_sampled_lifetime']

    # On récupère le nombre d'épisodes effectués
    episode_count += results['env_runners']['num_episodes']

    # On récupère le gain moyen au cours des épisodes effectués pendant le dernier appel à train()
    toPlot.append(results['env_runners']['episode_return_mean'])
    ordonnees.append(episode_count)

algo.stop()

plt.plot(ordonnees, toPlot)
plt.show()
ray.shutdown()
```

DQN de RLlib

Exploitation

```
module = algo.get_module()

taille_couloir = 4

env = Couloir({
    'n_states': taille_couloir
})

episode_return = 0.0
done = False
obs, info = env.reset()

while not done:
    env.render()

    # On code l'observation
    one_hot_obs = np.array([0 for _ in range(taille_couloir)], dtype = np.float32)
    one_hot_obs[int(obs)] = 1
    obs_batch = th.from_numpy(one_hot_obs).unsqueeze(0)

    # Récupération de la sortie du réseau
    model_outputs = module.forward_inference({'obs': obs_batch})

    # Récupération de l'action à effectuer
    action_dist_params = model_outputs["actions"][0].numpy()

    greedy_action = action_dist_params

    # Application de l'action
    obs, reward, terminated, truncated, info = env.step(greedy_action)

    episode_return += reward
    done = terminated or truncated
env.render()
print(f"Récompense obtenue au cours de l'épisode : {episode_return}")

+---+---+---+---+
| X ||  ||  || |
+---+---+---+---+
+---+---+---+---+
|  || X ||  || |
+---+---+---+---+
+---+---+---+---+
|  ||  || X || |
+---+---+---+---+
+---+---+---+---+
|  ||  ||  || X |
+---+---+---+---+
Récompense obtenue au cours de l'épisode : 8.0
```

4/Paramétrage et environnement

- Les différents paramètres du QLearning
- Ceux de DQN
- Comparaison des performances
- Changer l'environnement

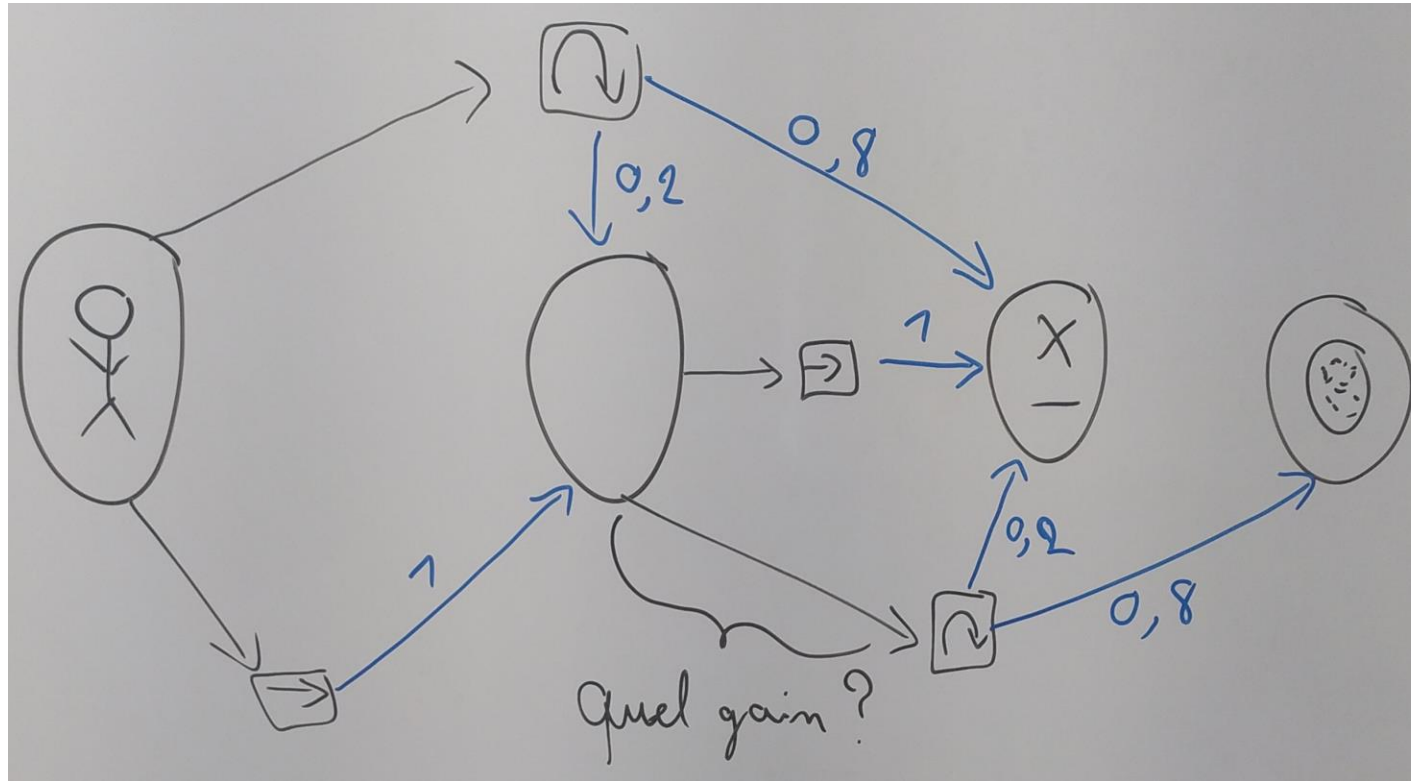
QLearning les paramètres

En 2 parties :

- paramètres pour actualiser la table
- paramètres d'exploration

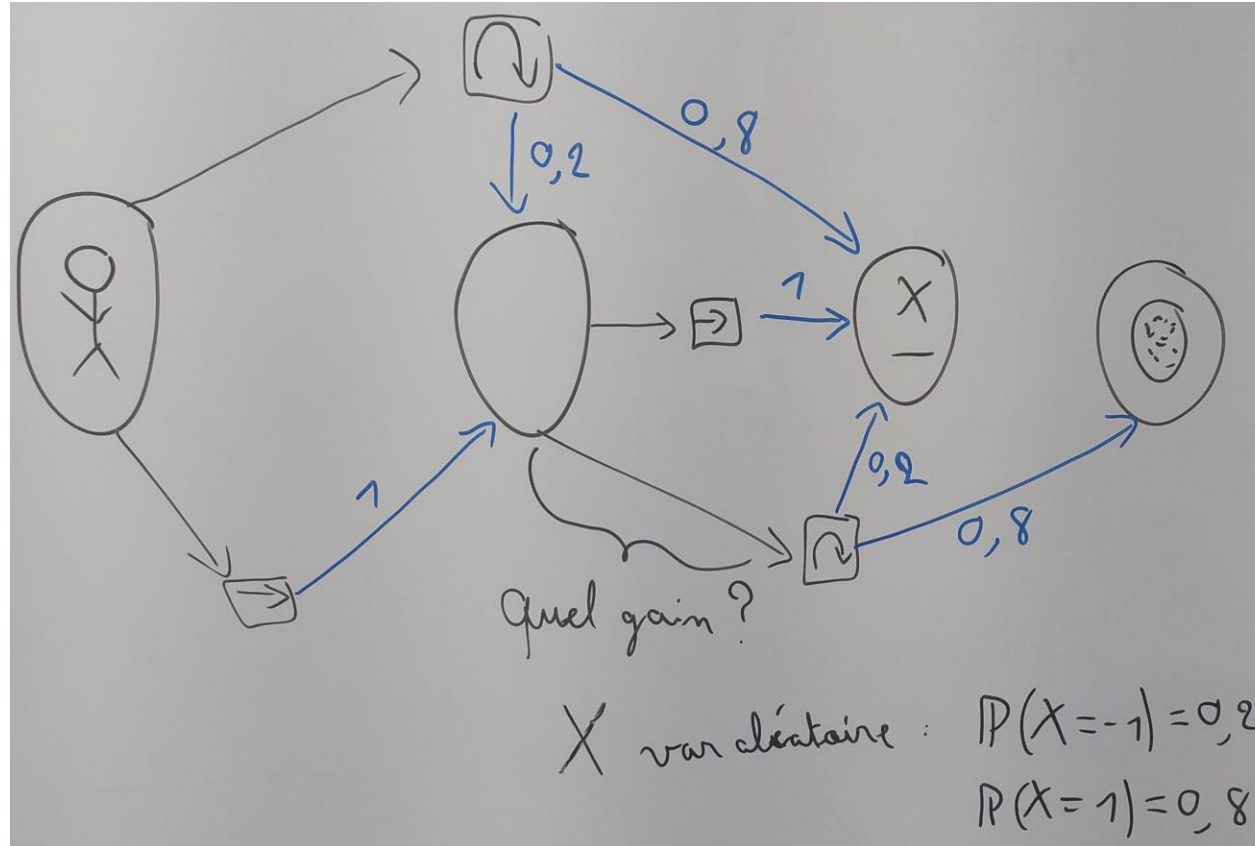
QLearning - Paramètres

Actualiser la table



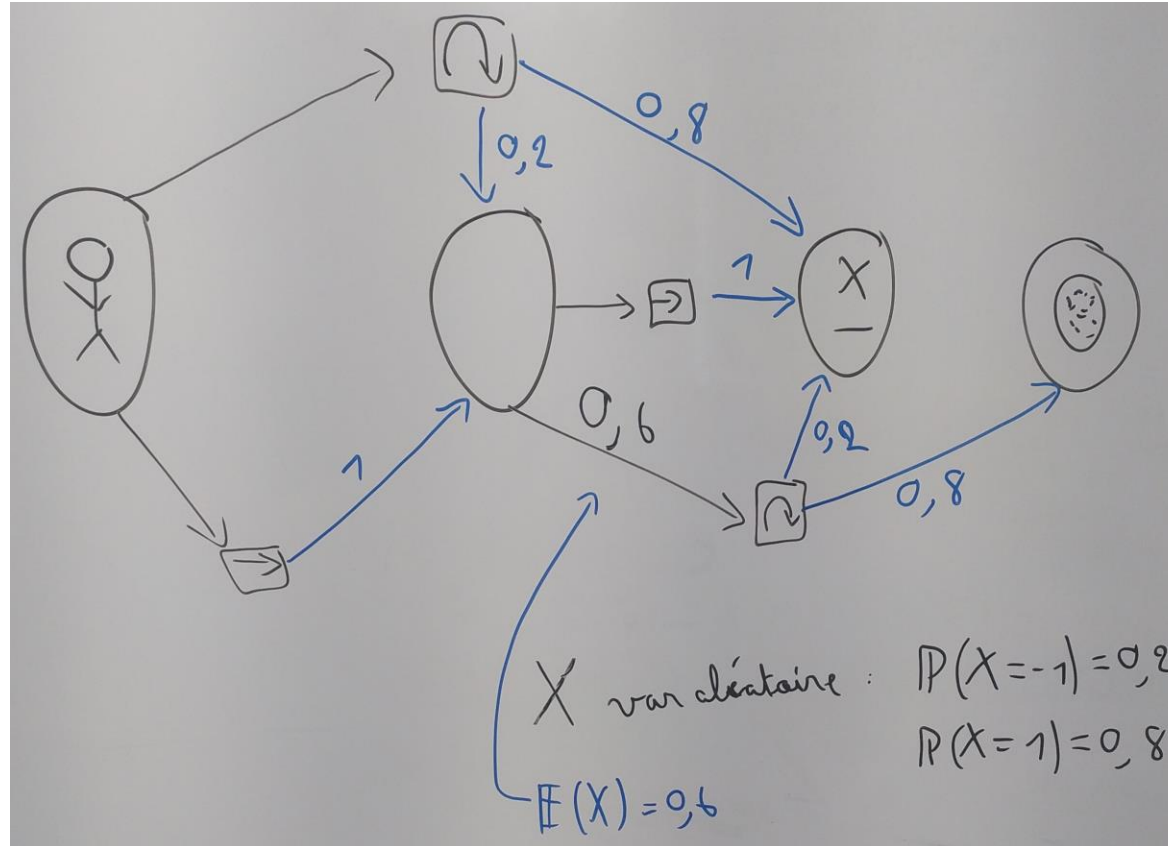
QLearning - Paramètres

Actualiser la table



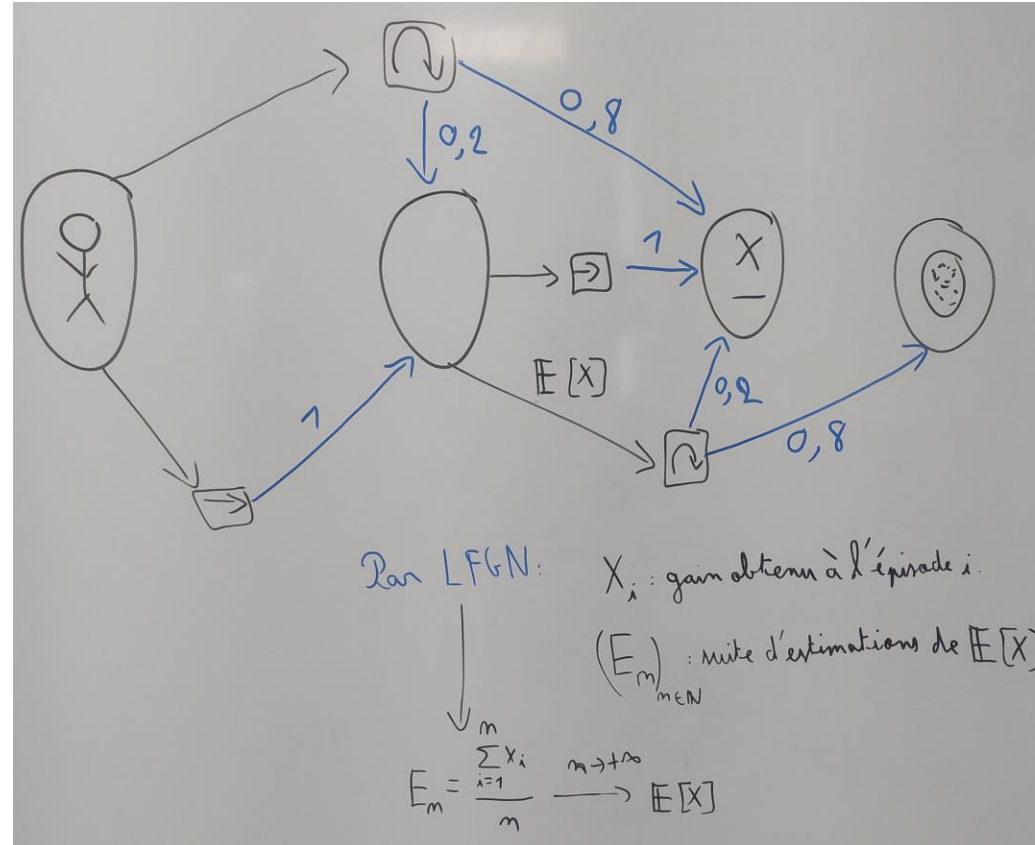
QLearning - Paramètres

Actualiser la table



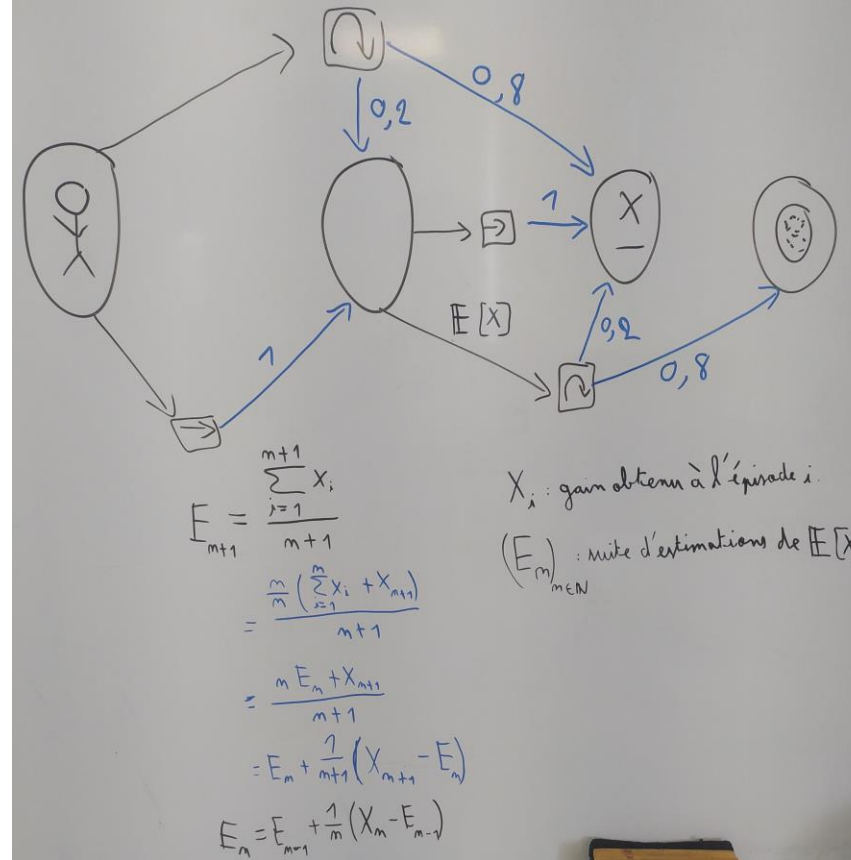
QLearning - Paramètres

Actualiser la table



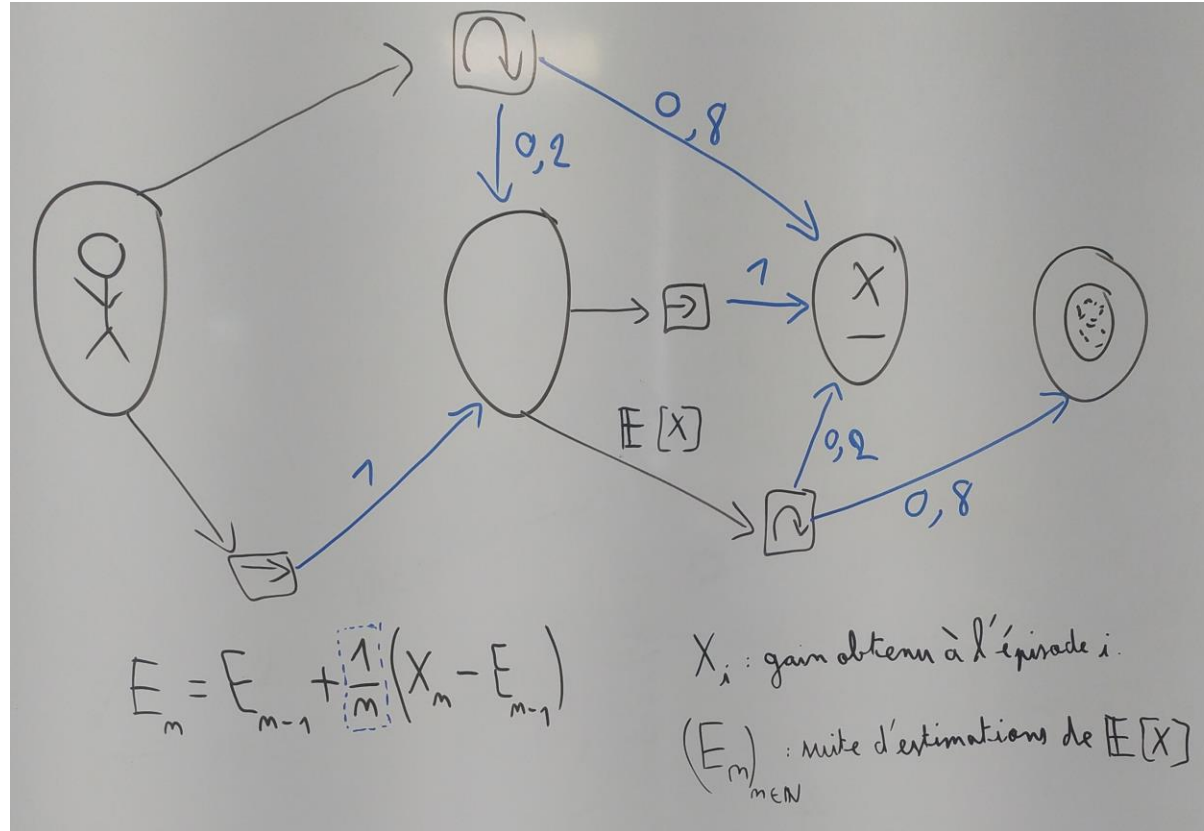
QLearning - Paramètres

Actualiser la table



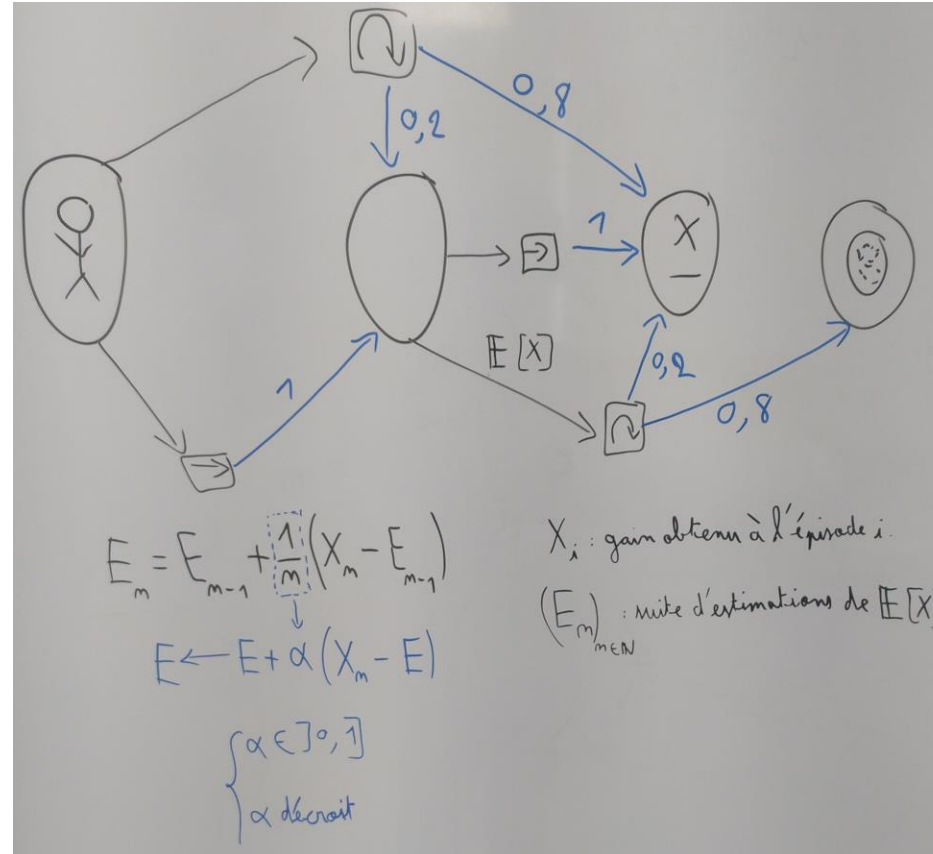
QLearning - Paramètres

Actualiser la table



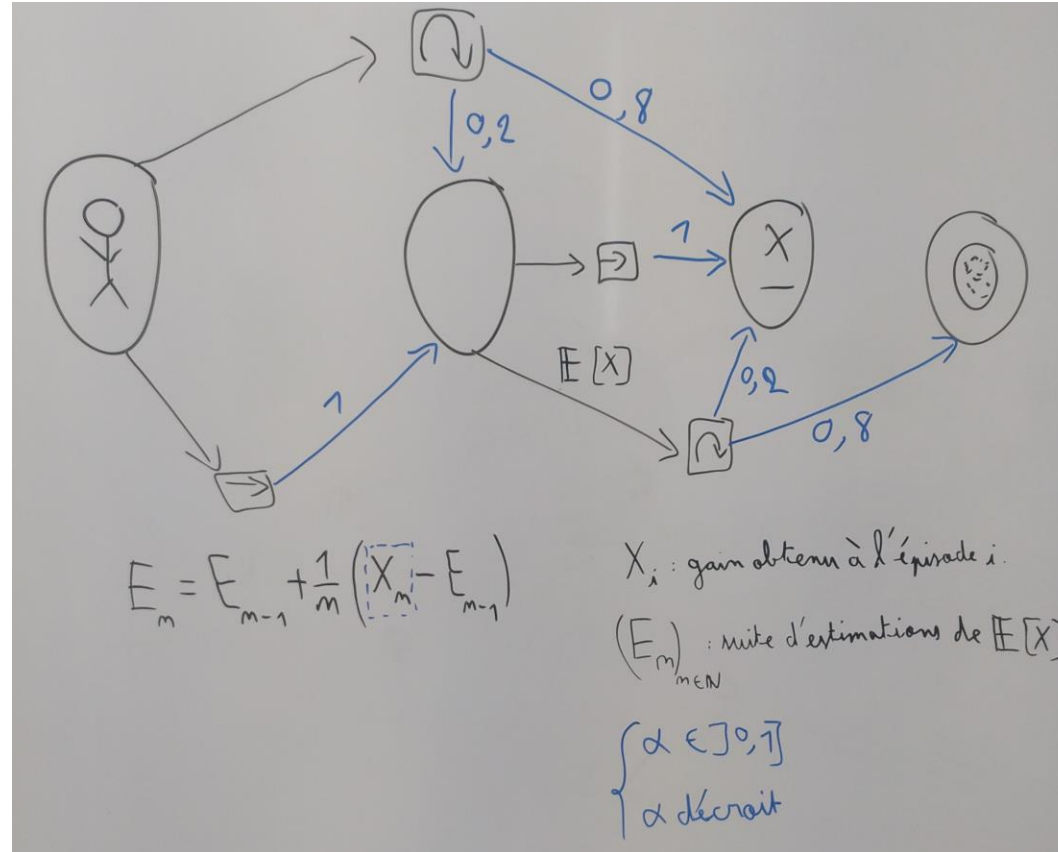
QLearning - Paramètres

Actualiser la table



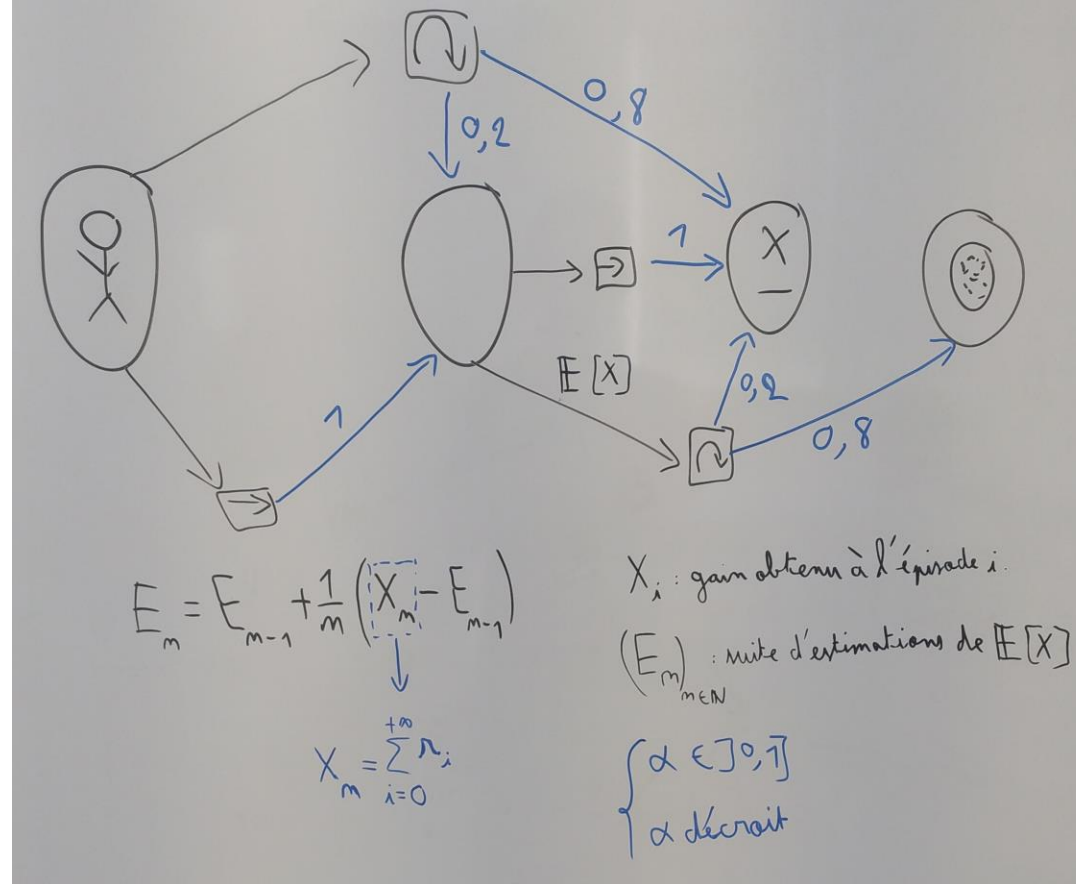
QLearning - Paramètres

Actualiser la table



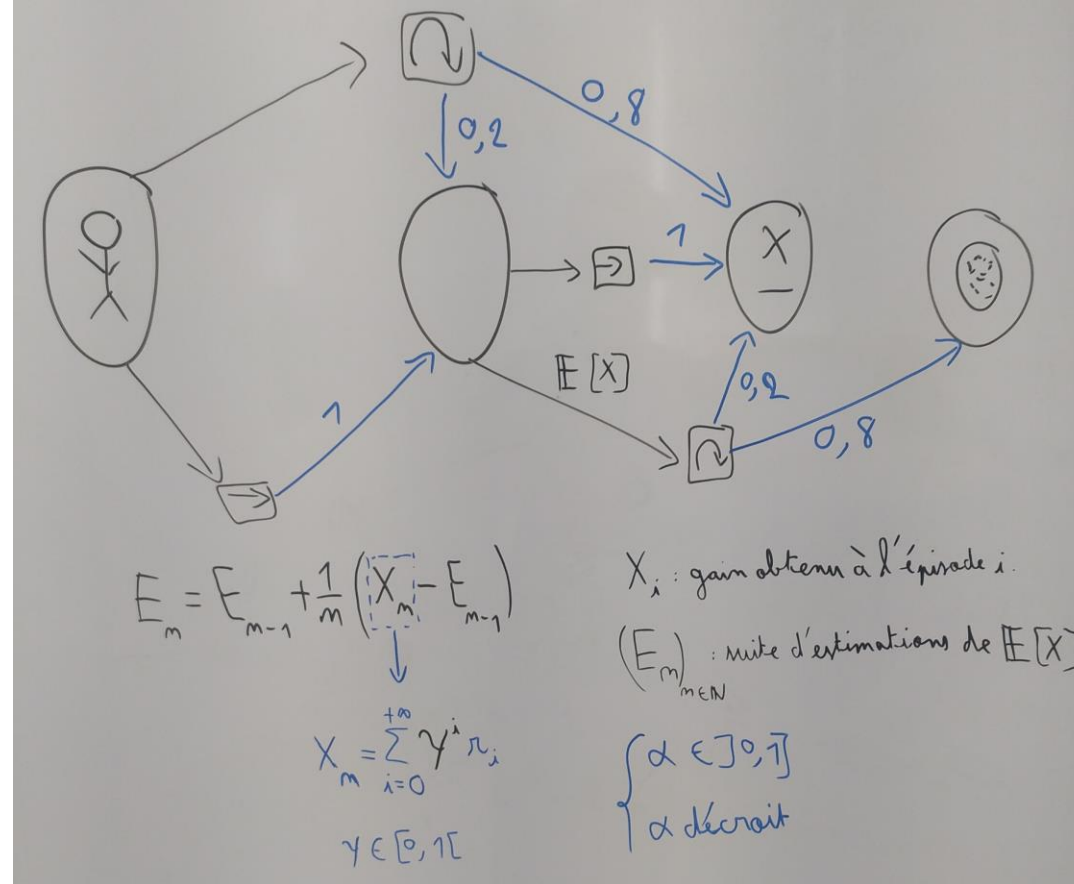
QLearning - Paramètres

Actualiser la table



QLearning - Paramètres

Actualiser la table



QLearning - Paramètres

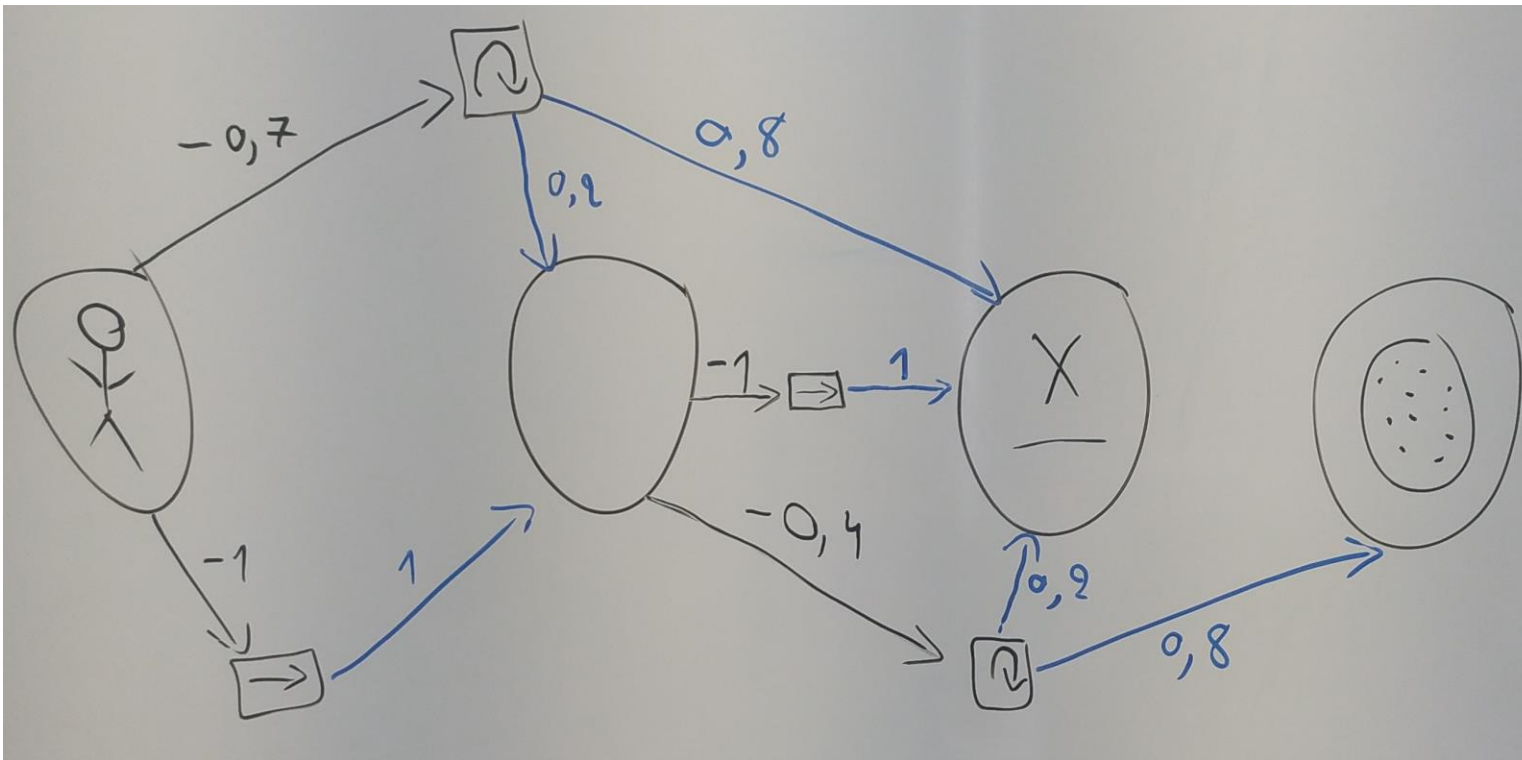
Actualiser la table

2 paramètres :

- **alpha** dans $]0, 1]$ et décroît
- **gamma** dans $[0, 1[$

QLearning - Paramètres

Exploration



QLearning - Paramètres

Exploration

3 paramètres :

- **eps_start** : la valeur de epsilon en début d'entraînement
- **eps_end** : la valeur de epsilon après décroissance
- **eps_fraction** : la fraction de l'entraînement au cours de laquelle epsilon décroît de façon linéaire

QLearning – Couloir

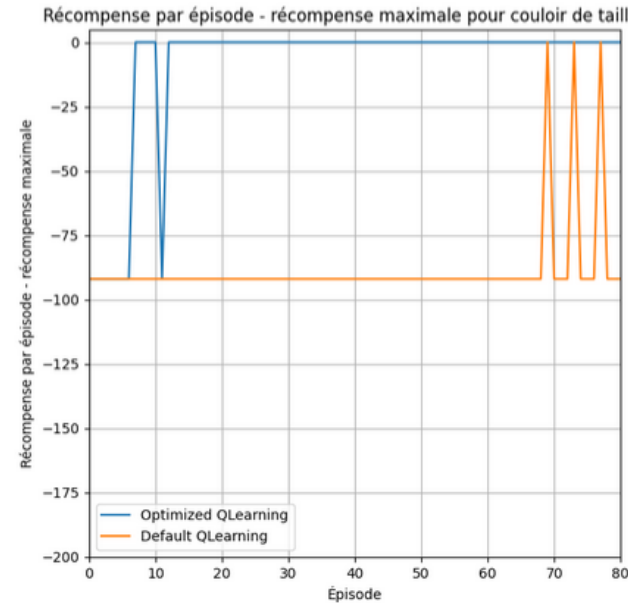
Comparaison

Par défaut :

- Alpha = 0.1
- Gamma = 0.9
- Eps_start = 0.9

Après gridSearch :

- Alpha = 0.9
- Gamma = 0.8
- Eps_start = 0.7



DQN - Paramètres Communs

Certains paramètres de DQN sont communs avec le QLearning

QLearning	DQN
alpha	learning_rate
gamma	gamma
eps_start	exploration_initial_eps
eps_end	exploration_final_eps
eps_fraction	exploration_fraction

DQN - Paramètres

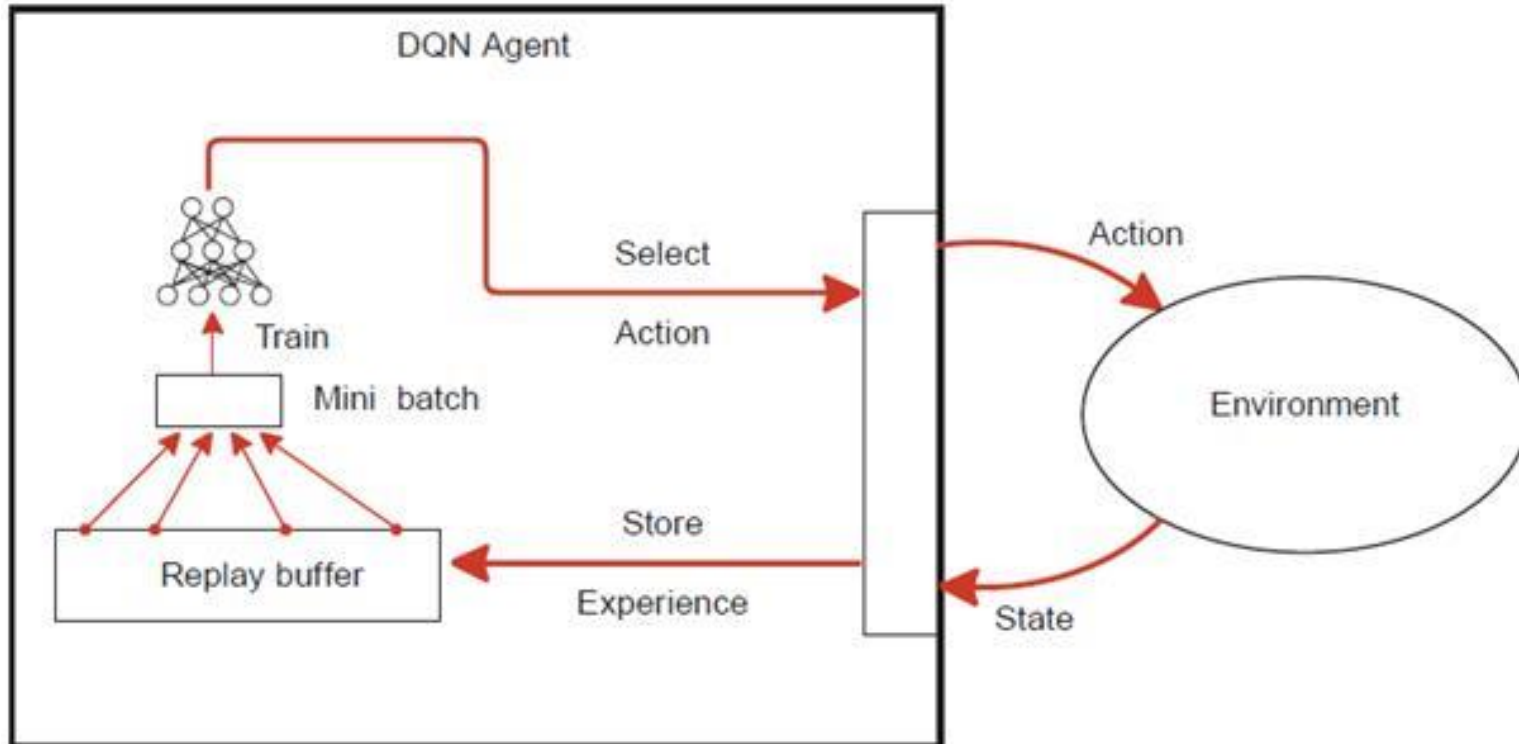
Réseau

Le réseau introduit 4 nouveaux paramètres :

- `net_arch`
- `buffer_size`
- `batch_size`
- `target_update_interval`

DQN - Paramètres

Réseau : buffer & batch



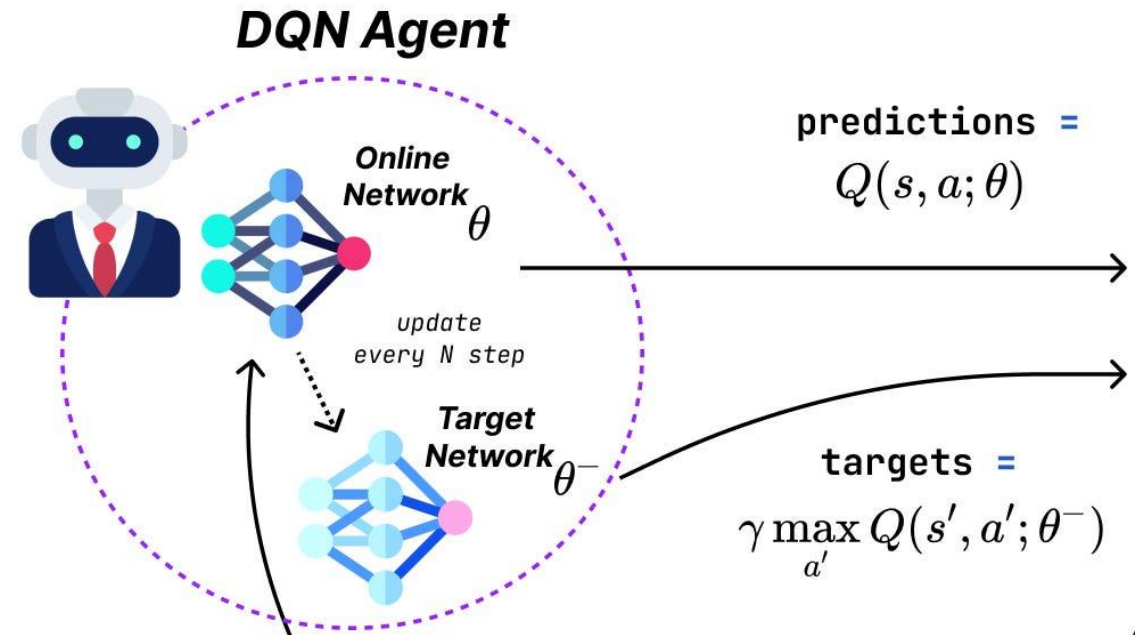
$$E_m = E_{m-1} + \frac{1}{m} (X_m - E_{m-1})$$

$$X_m = \sum_{i=0}^{+\infty} \gamma^i r_i$$

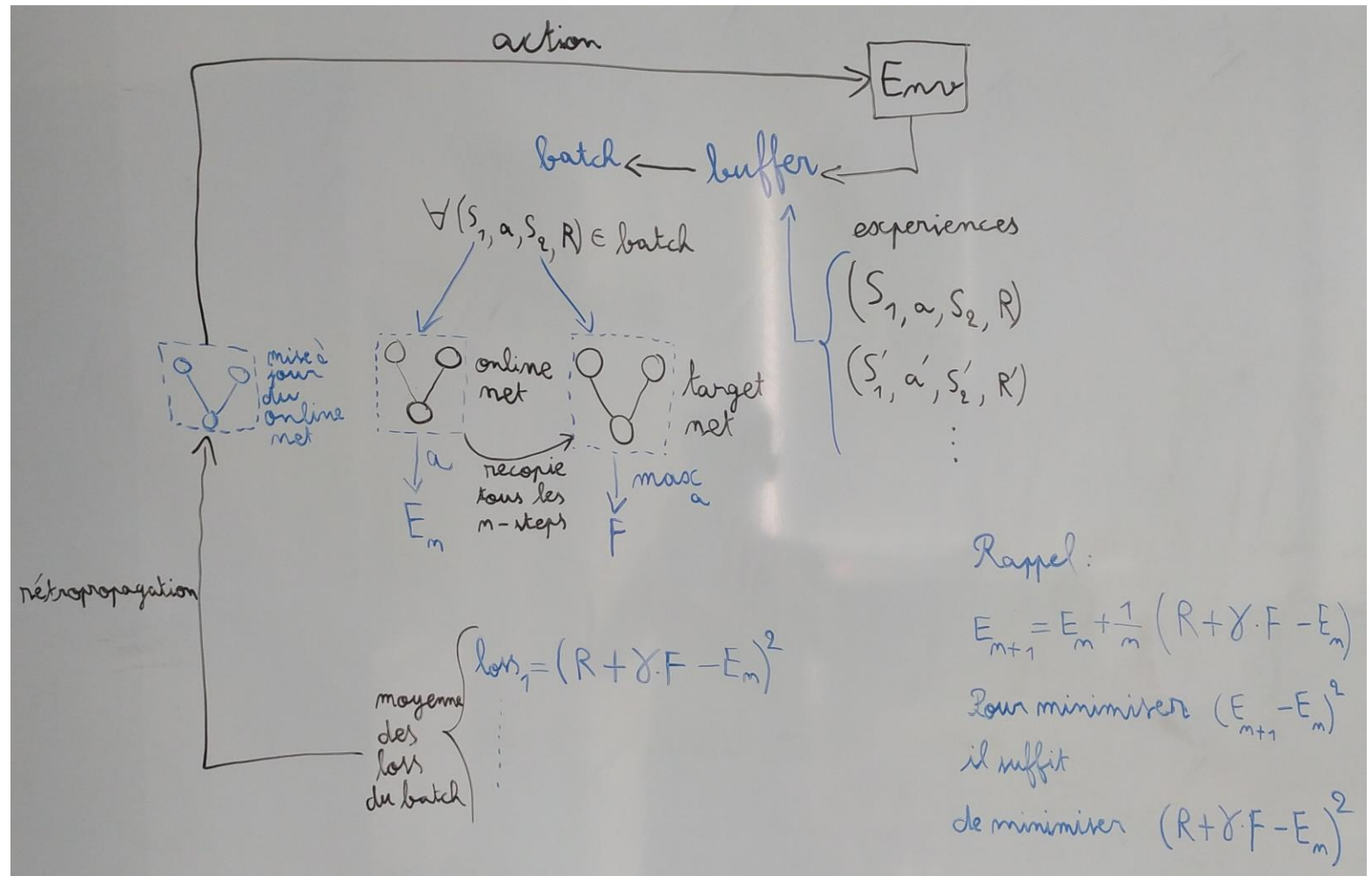
DQN - Paramètres

Réseau

target network



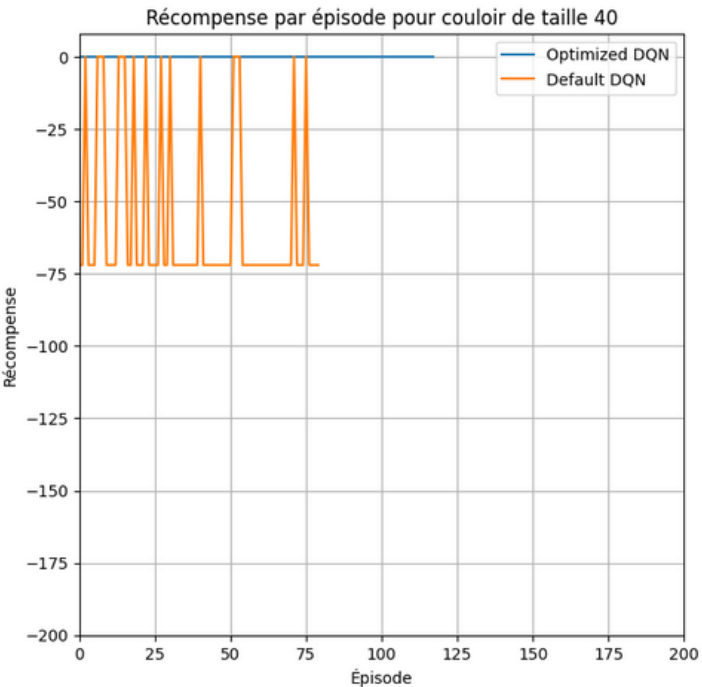
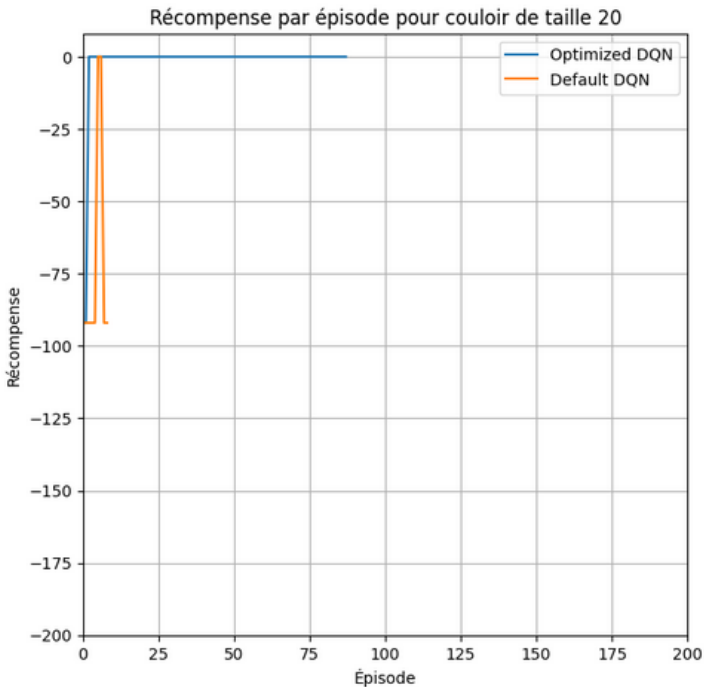
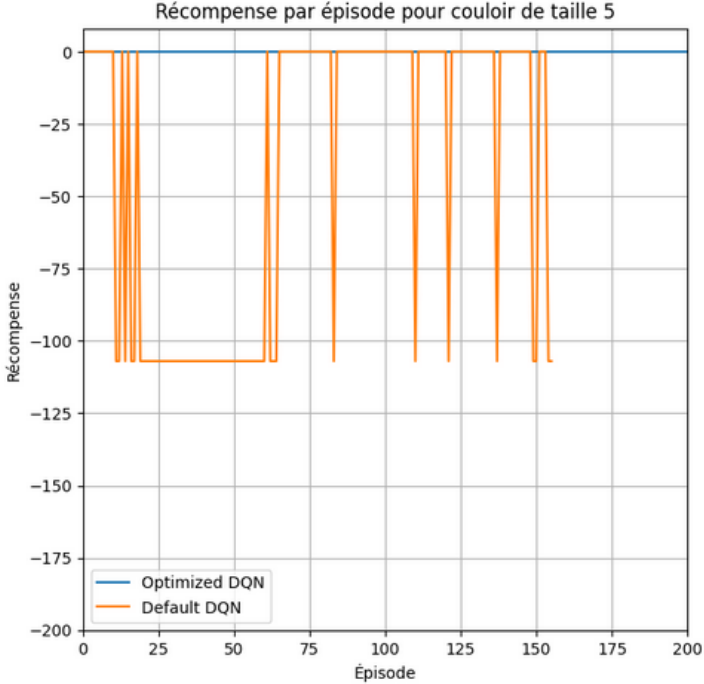
DQN - Schéma



DQN - Couloir

Comparaison

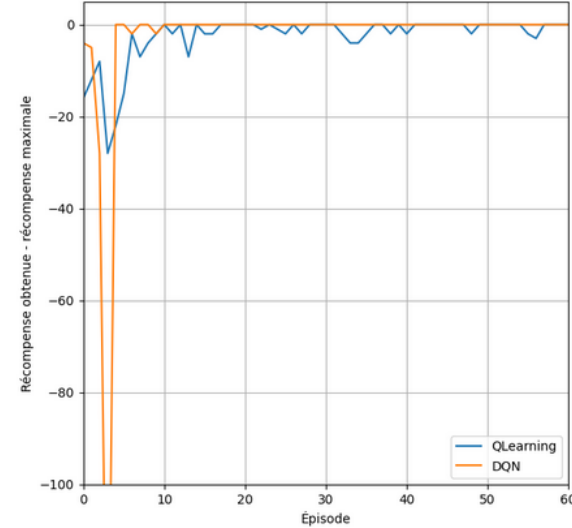
Paramètre	Par défaut	Paramétré
Net_arch	[64, 64]	[2]
Update_freq	500	5



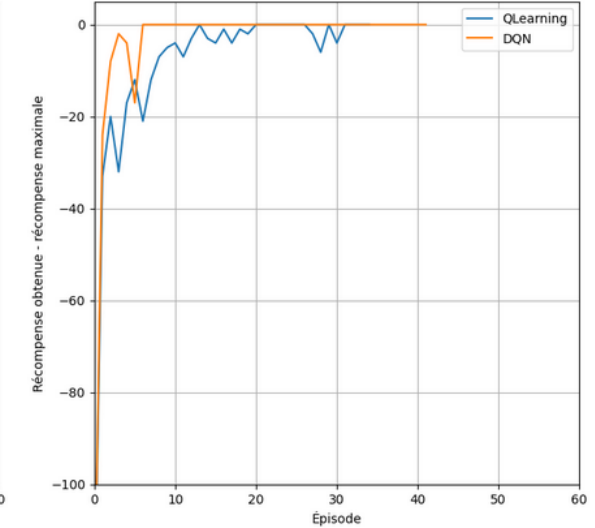
Comparaison couloir

QLearning /
DQN stable
baselines

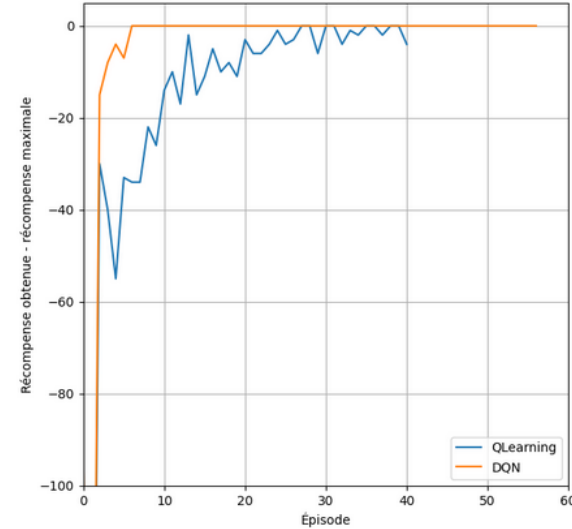
Différence entre récompense obtenue et récompense optimale pour couloir de taille 20



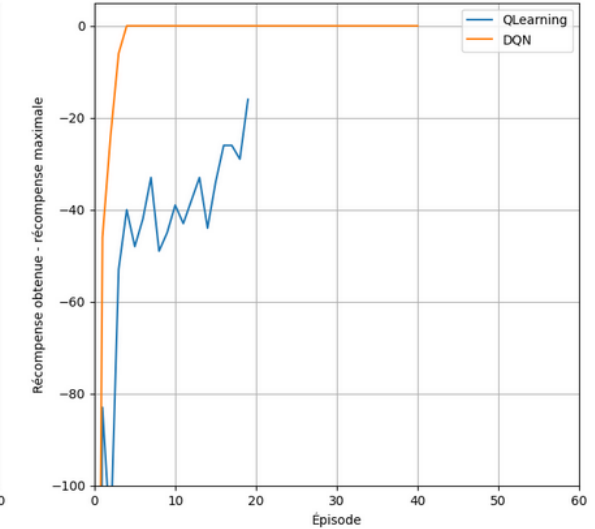
Différence entre récompense obtenue et récompense optimale pour couloir de taille 20



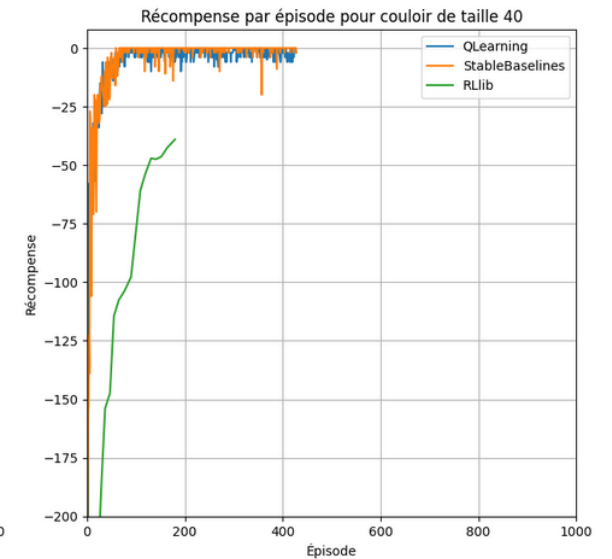
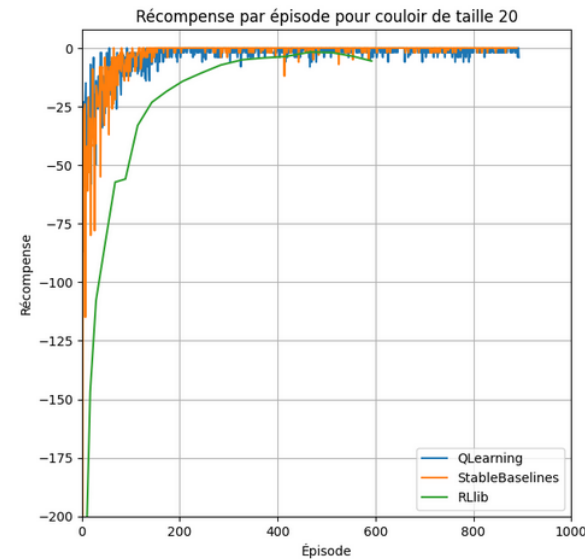
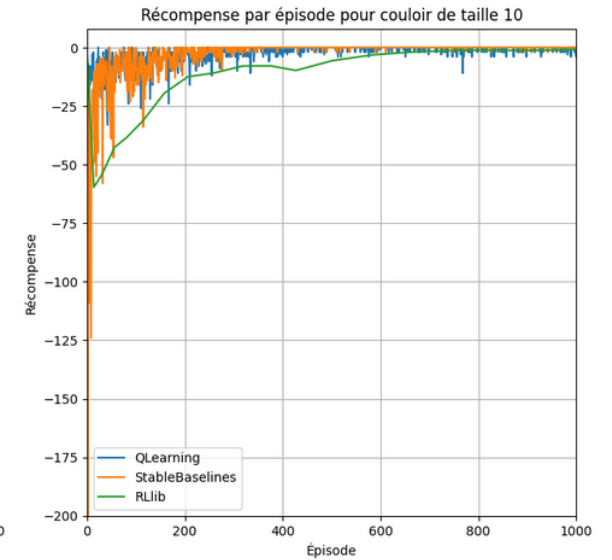
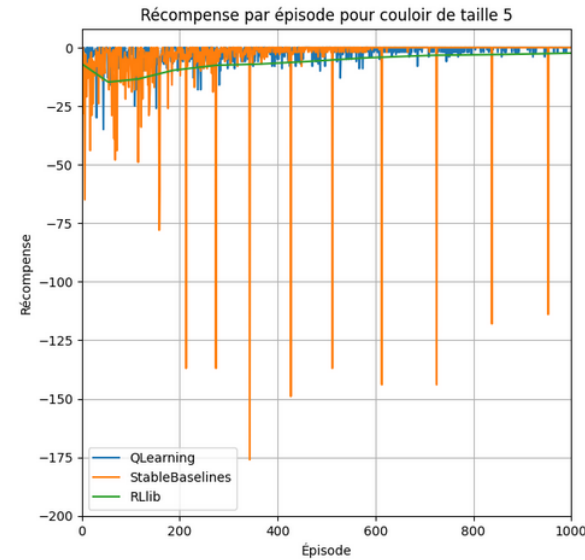
Différence entre récompense obtenue et récompense optimale pour couloir de taille 40



Différence entre récompense obtenue et récompense optimale pour couloir de taille 40



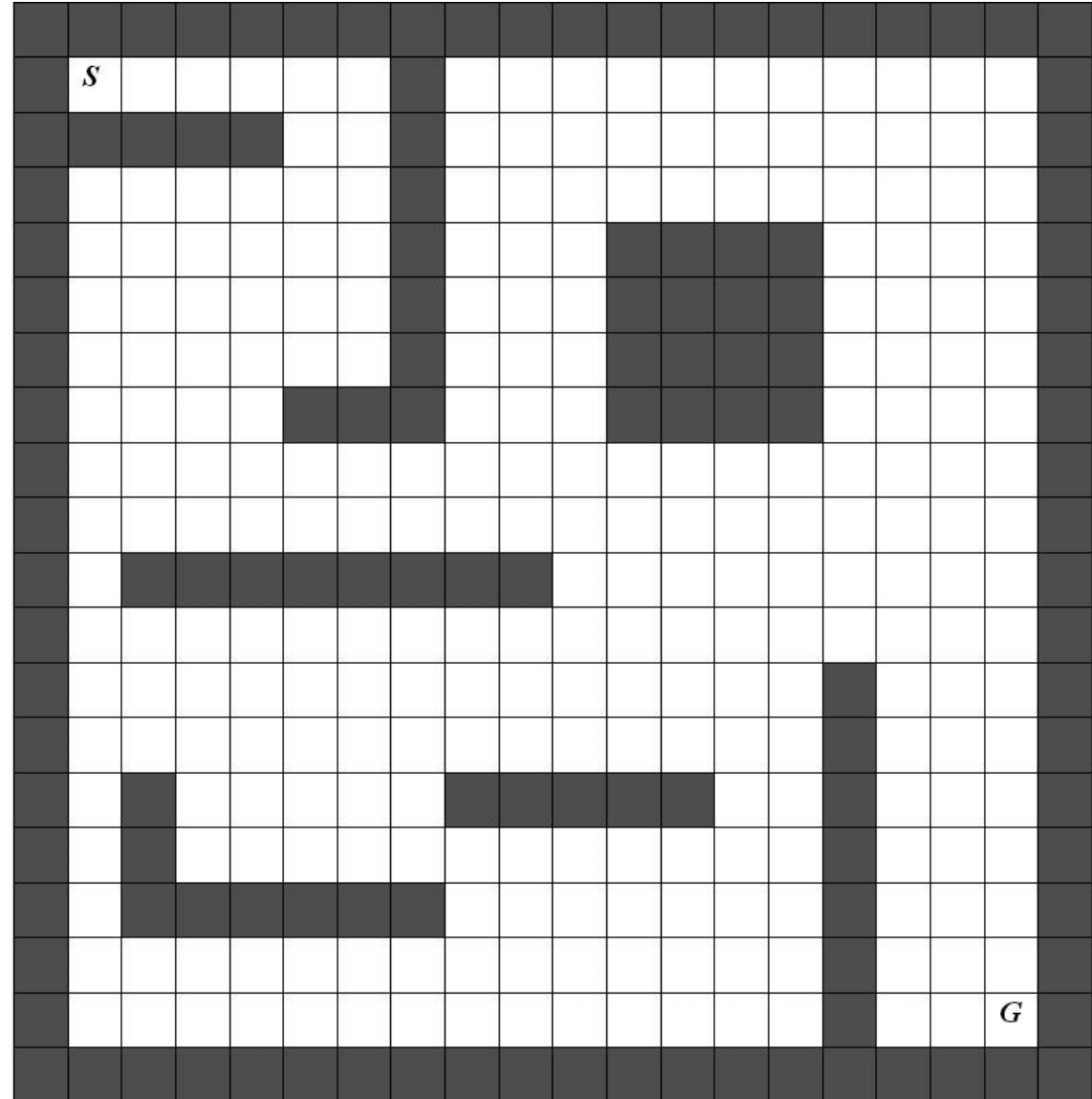
Comparaison couloir les 3 algorithmes



Nouvel Environnement Labyrinthe

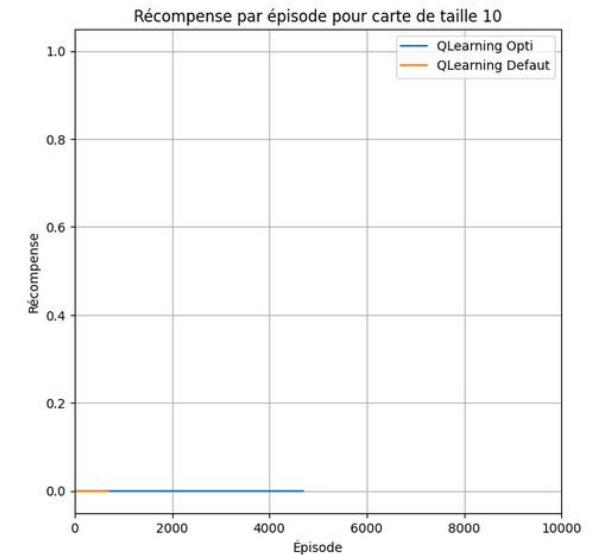
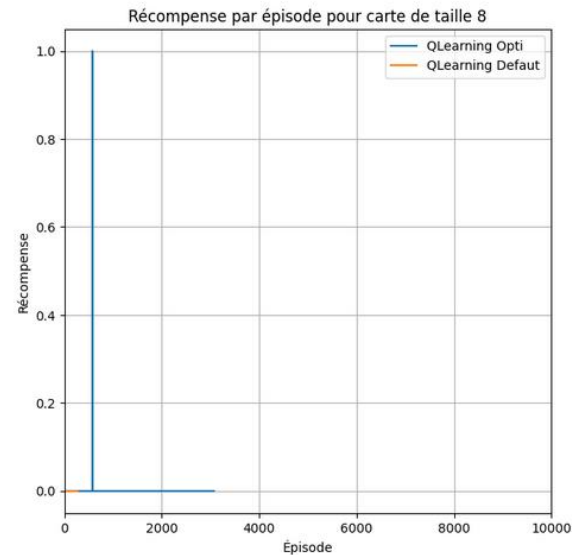
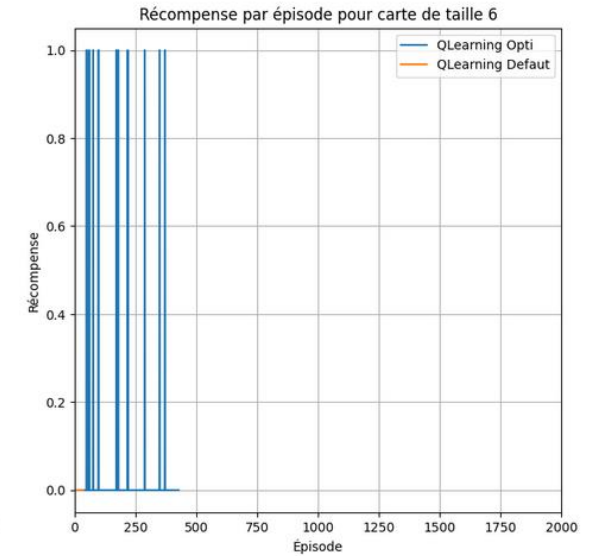
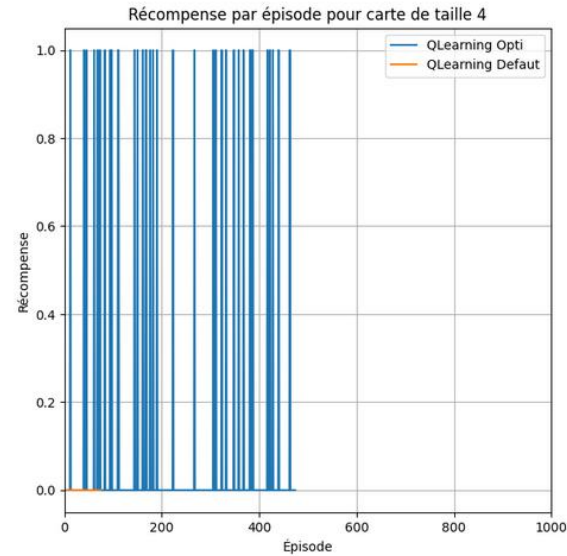
Observation :
Position

Récompense :
1 si sortie
0 sinon



Résultats QLearning avec cet environnement

- Observation :
Position
- Récompense :
- 1 si sortie
- 0 sinon



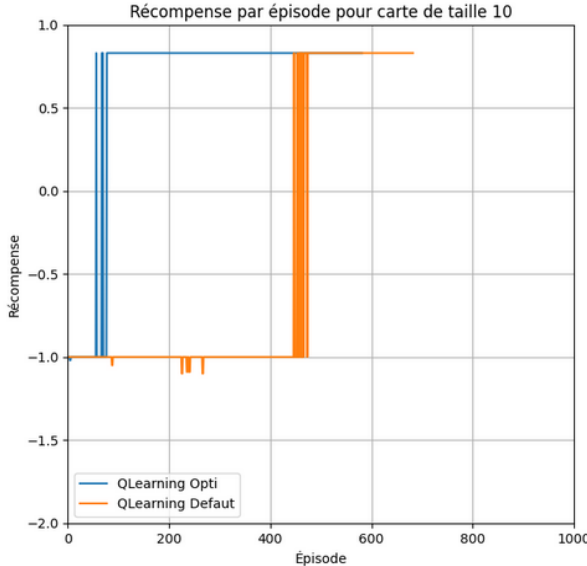
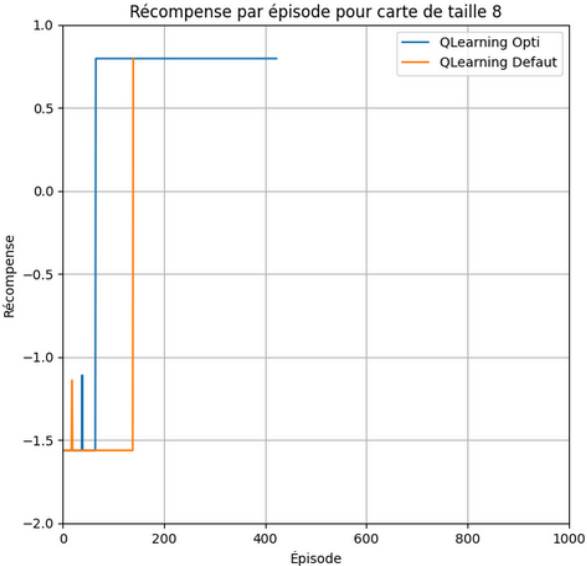
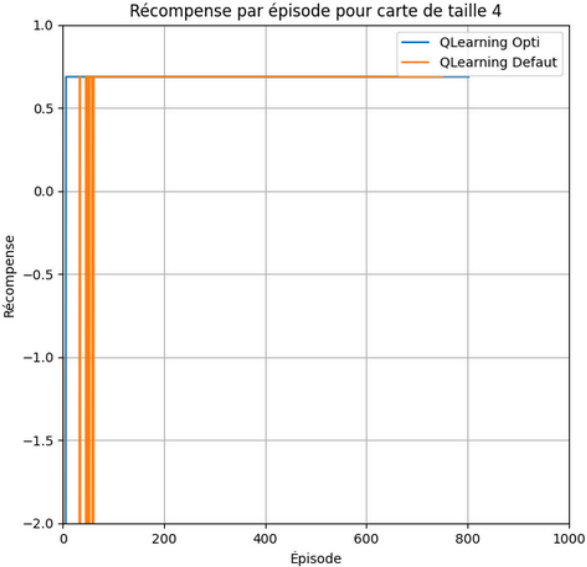
Le Labyrinthe contre-attaque

Nouvelle distribution de récompense !

- $-1/\text{nbCases}$ pour chaque déplacement
- -1 pour les trous
- $+1$ pour la sortie

QLearning pour labyrinthe 2

Paramètre	Par défaut	Paramétré
alpha	0.1	1
gamma	0.9	0.99
eps_start	0.9	0.05
eps_end	0.05	0.05

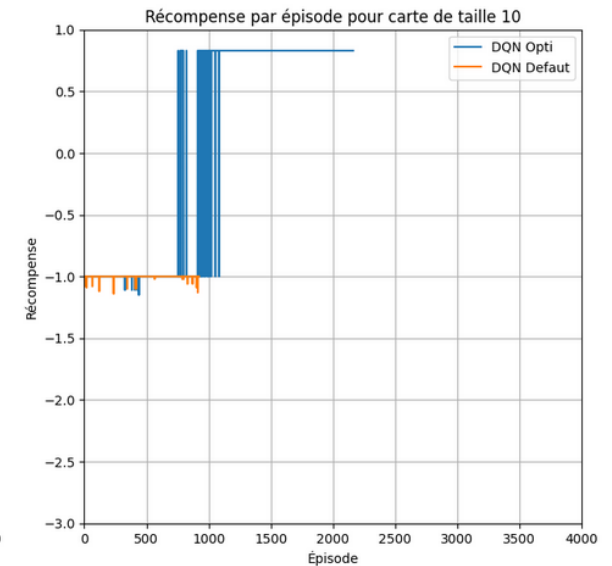
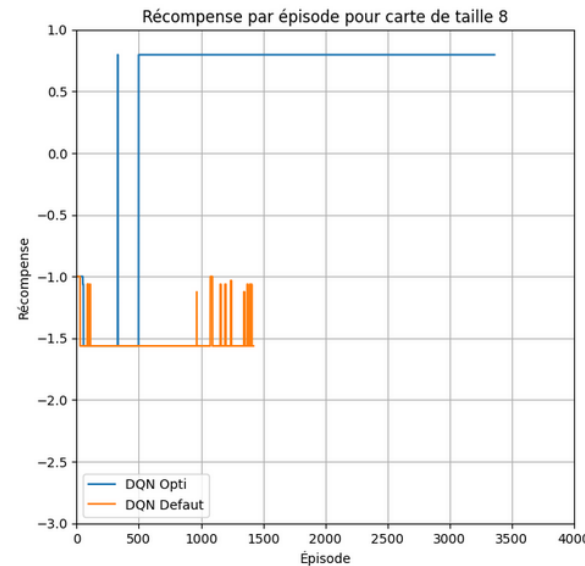
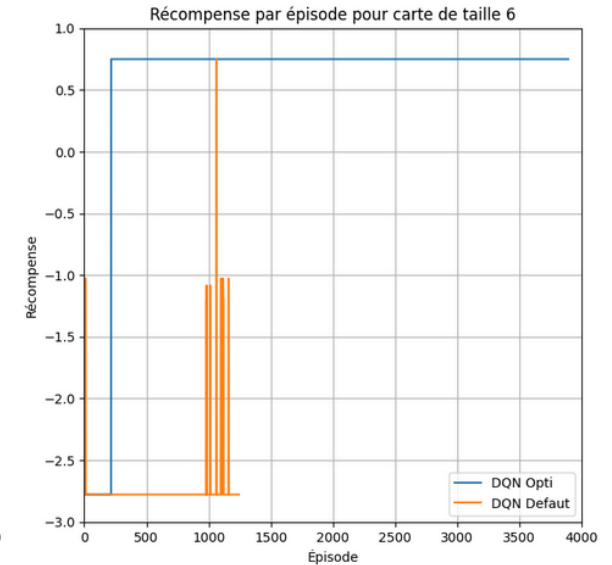
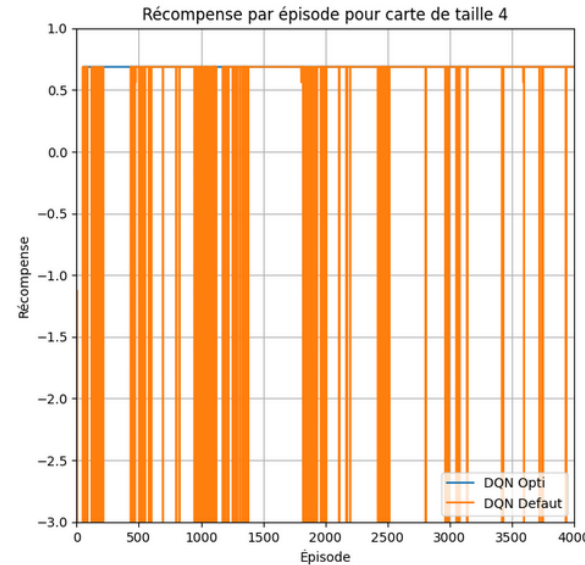


DQN Stable

Baselines pour

labyrinthe 2

Paramètre	Par défaut	Paramétré
alpha	0.01	0.0001
gamma	0.99	0.9
net_arch	[64, 64]	[32] / [32, 32] pour T10
update_freq	500	50
buffer_size	500	10000
batch_size	32	1000



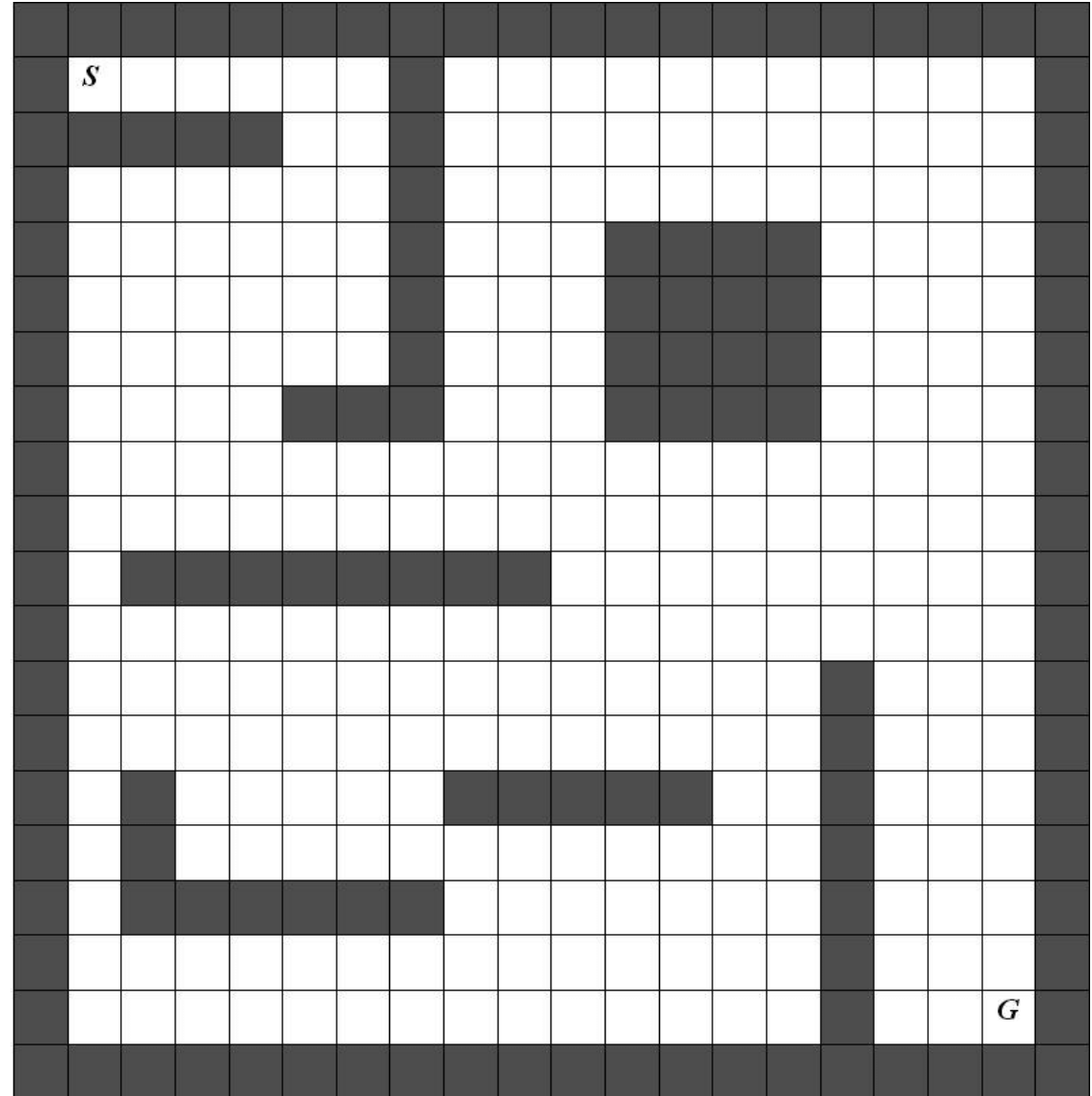
Retour vers le labyrinthe

Observation :

Position

Cases Adjacentes

Même récompenses que
labyrinthe 2



Labyrinthe 3

MultiDiscrete

spaces

```
class FrozenLake3Env(gymnasium.Env) :
    def __init__(self, options = None) :
        super(FrozenLake3Env, self).__init__()
        self.map = options['lake_map']
        self.width = len(self.map[0])
        self.height = len(self.map)
        self.cell_values = {'F' : 0, 'S' : 0, 'H' : 1, 'G' : 2, 'W' : 3}
        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.MultiDiscrete([self.width * self.height, 4, 4, 4, 4])
        self.state = 0

        for x in range(self.width) :
            for y in range(self.height) :
                if self.map[y][x] == 'S' :
                    self.state = self._coordinateToState(x, y)
                    break

    def reset(self, seed = None, options = None) :
        super().reset(seed = seed)
        self.state = 0

        for x in range(self.width) :
            for y in range(self.height) :
                if self.map[y][x] == 'S' :
                    self.state = self._coordinateToState(x, y)
                    break

    left, down, right, up = self._getNeighbors(self.state)
    return np.array([self.state, left, down, right, up]), {}

    def step(self, action) :
        x, y = self._stateToCoordinate(self.state)
        if action == 0 : # Left
            if x > 0 :
                x -= 1
        elif action == 1 : # Down
            if y < self.height - 1 :
                y += 1
        elif action == 2 : # Right
            if x < self.width - 1 :
                x += 1
        elif action == 3 : # Up
            if y > 0 :
                y -= 1

        self.state = self._coordinateToState(x, y)

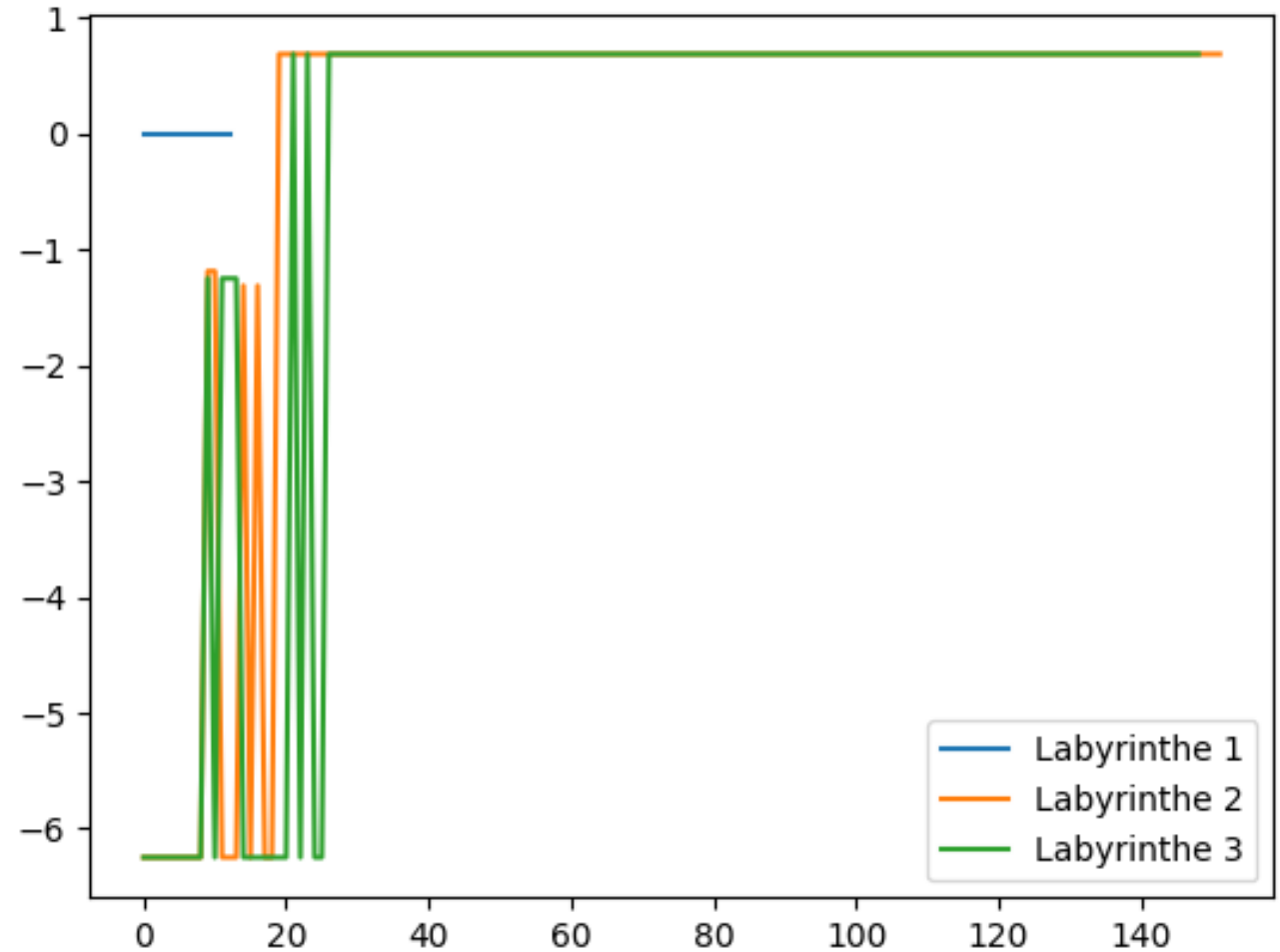
        result = self.map[y][x]
        done = result in ['H', 'G']
        if result == 'G' :
            reward = 1
        elif result == 'H' :
            reward = -1
        else :
            reward = -1/(self.width * self.height)

        left, down, right, up = self._getNeighbors(self.state)
        return np.array([self.state, left, down, right, up]), reward, done, False, {}
```

QLearning labyrinthe 3

Nous remarquons aucune
différence de performances pour
le QLearning entre les
labyrinthes 2 et 3

Aucune différence en raison du
nombre de situations observables

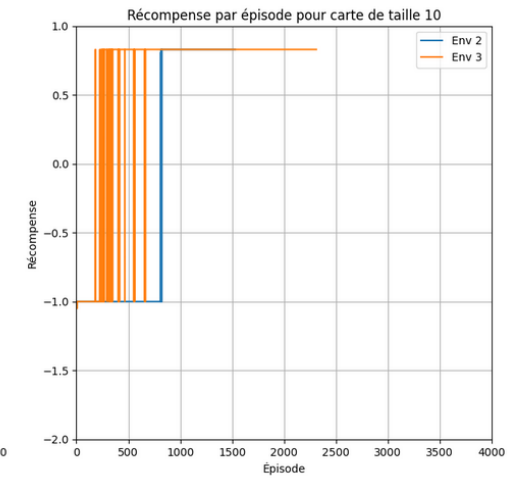
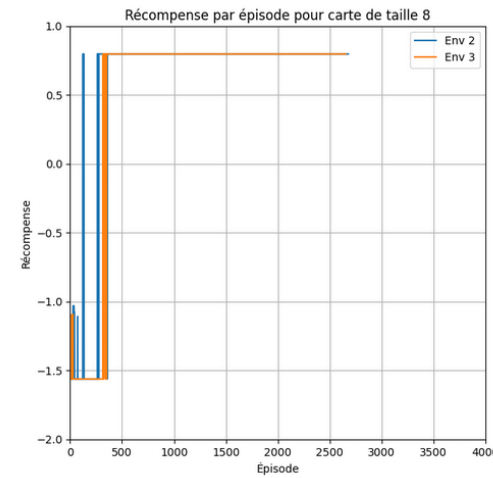
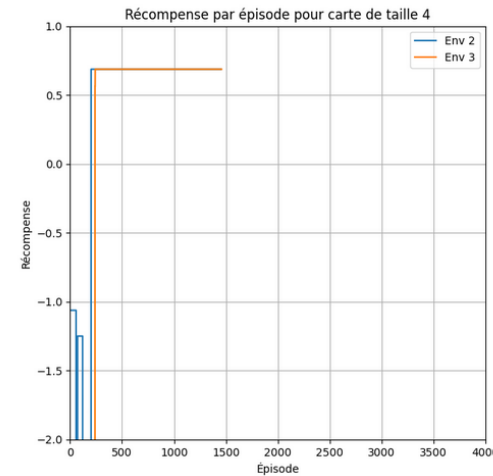


DQN

labyrinthe

3

Les paramètres optimaux
sont indentiques au
labyrinthe 2



Labyrinthe 4

On code la position dans le réseau DQN à l'aide de deux neurones.

Pour faire cela on utilise `spaces.Box`

Comme on souhaite conserver nos observations il faut aussi utiliser `spaces.Dict`

```
class FrozenLake4Env(gymnasium.Env) :
    def __init__(self, options = None) :
        super(FrozenLake4Env, self).__init__()
        self.map = options['lake_map']
        self.width = len(self.map[0])
        self.height = len(self.map)
        self.cell_values = {'F' : 0, 'S' : 0, 'H' : 1, 'G' : 2, 'W' : 3}
        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.Dict({
            'row_position' : spaces.Box(low = 0, high = self.width - 1, dtype = int),
            'col_position' : spaces.Box(low = 0, high = self.height - 1, dtype = int),
            'left_neighbor' : spaces.Discrete(4),
            'down_neighbor' : spaces.Discrete(4),
            'right_neighbor' : spaces.Discrete(4),
            'up_neighbor' : spaces.Discrete(4)
        })
        self.state = 0

        for x in range(self.width) :
            for y in range(self.height) :
                if self.map[y][x] == 'S' :
                    self.state = self._coordinateToState(x, y)
                    break

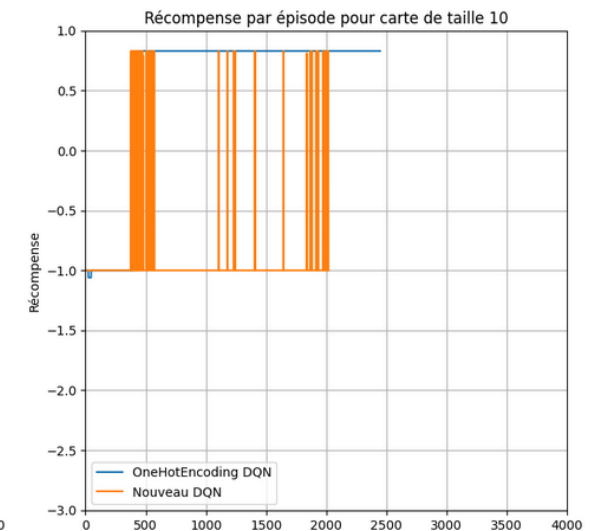
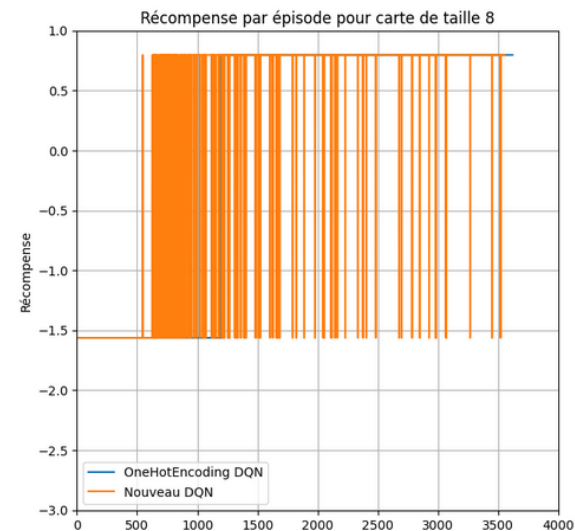
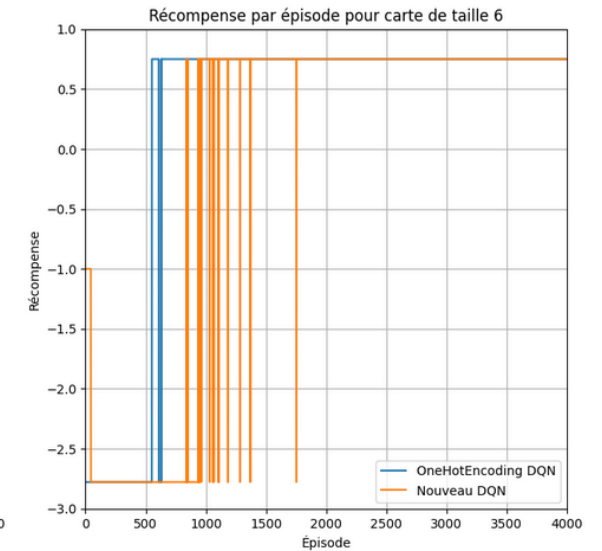
    def reset(self, seed = None, options = None) :
        super().reset(seed = seed)
        self.state = 0

        for x in range(self.width) :
            for y in range(self.height) :
                if self.map[y][x] == 'S' :
                    self.state = self._coordinateToState(x, y)
                    self.x = x
                    self.y = y
                    break

    left, down, right, up = self._getNeighbors(self.state)
    toReturn = {
        'row_position' : np.array([self.x]),
        'col_position' : np.array([self.y]),
        'left_neighbor' : left,
        'down_neighbor' : down,
        'right_neighbor' : right,
        'up_neighbor' : up
    }
    return toReturn, {}
```

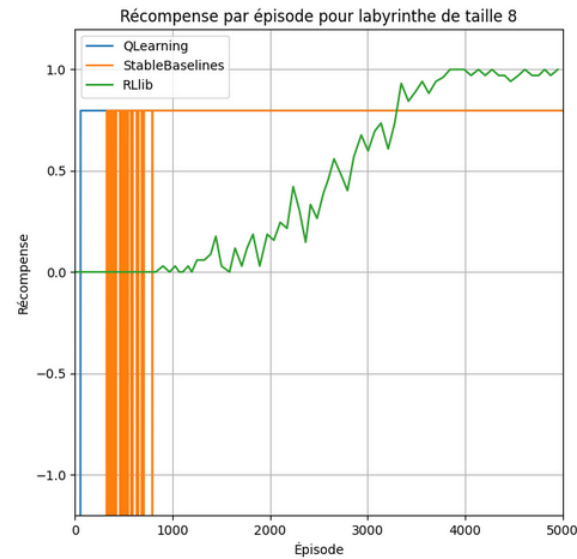
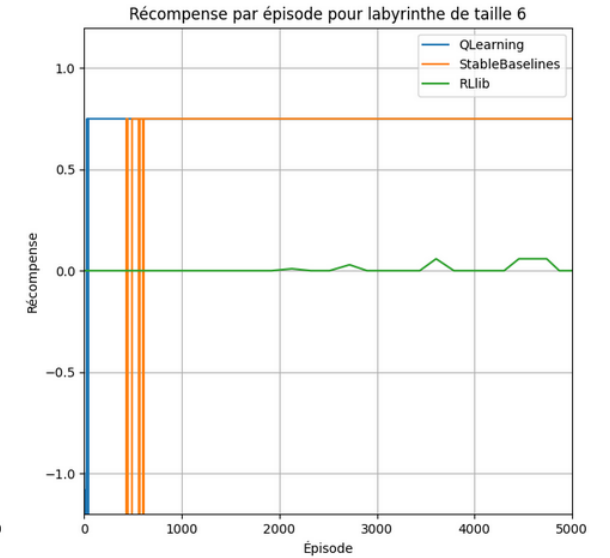
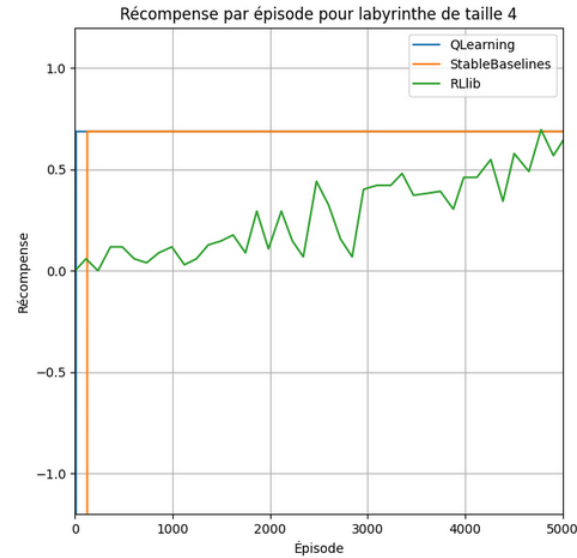
DQN pour labyrinthe 4

Les résultats sont très
mitigés.
Le one hot encoding est
plus efficace.



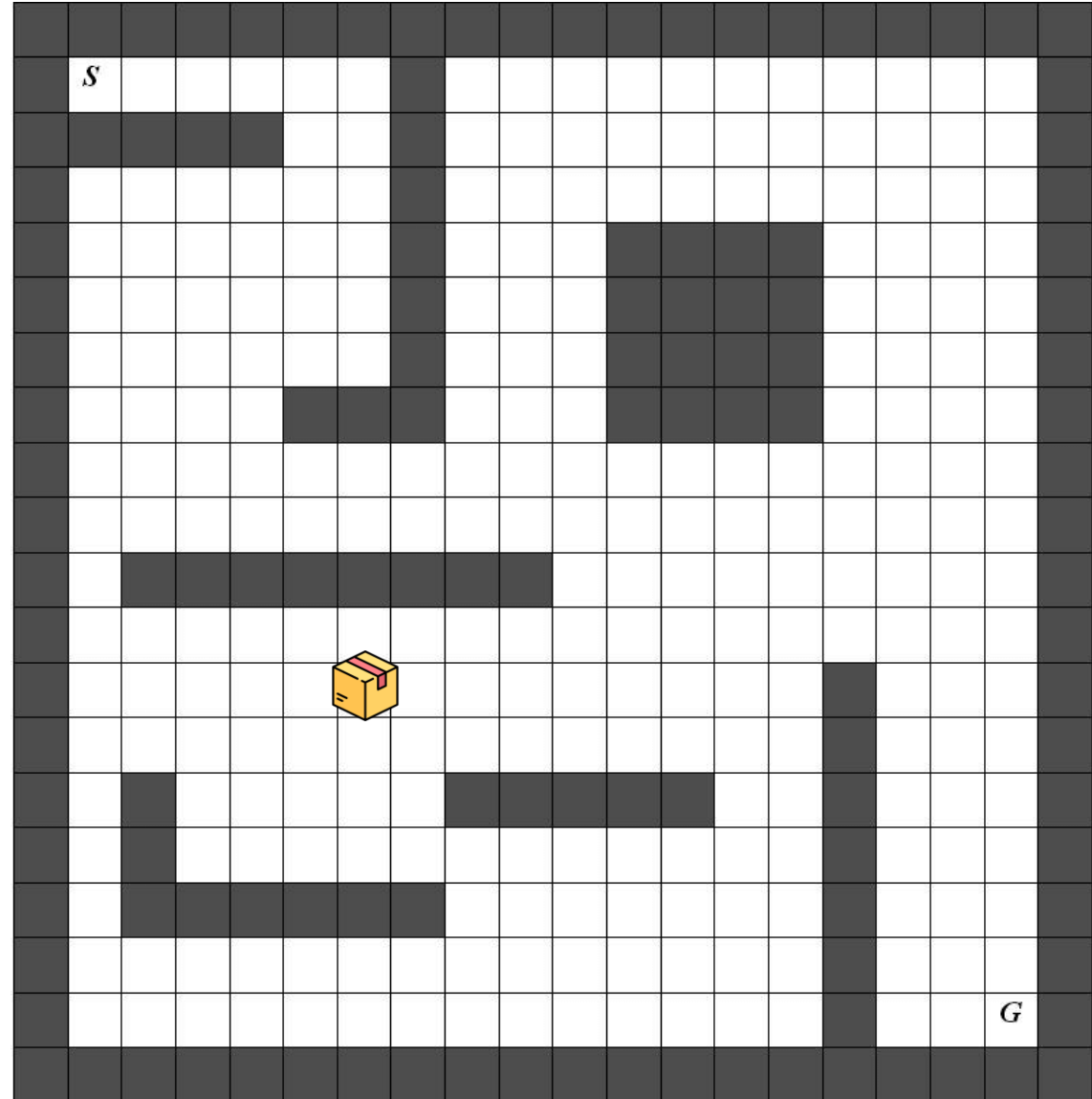
DQN

Résultats



Environnement Livraison

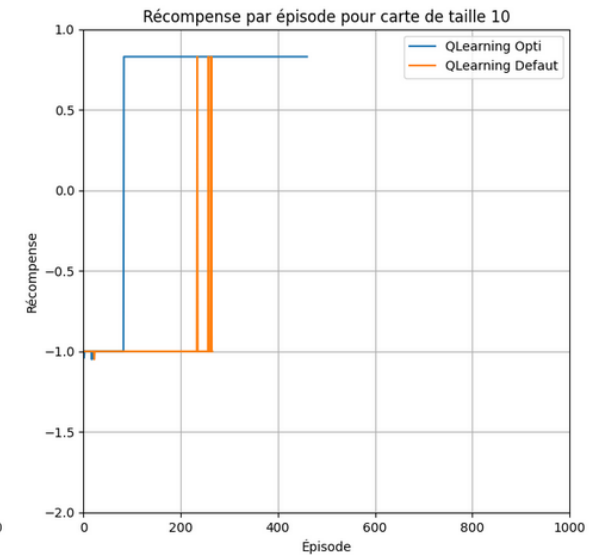
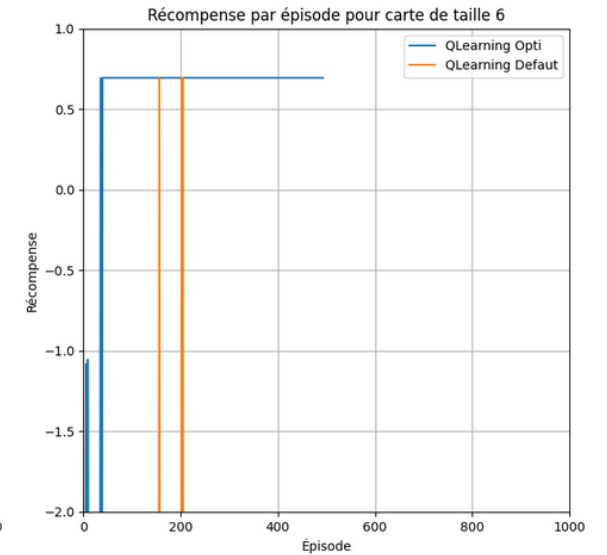
Il faut récupérer la boîte
avant d'atteindre la
sortie



Livraison - QLearning

Mêmes paramètres que pour Labyrinthe

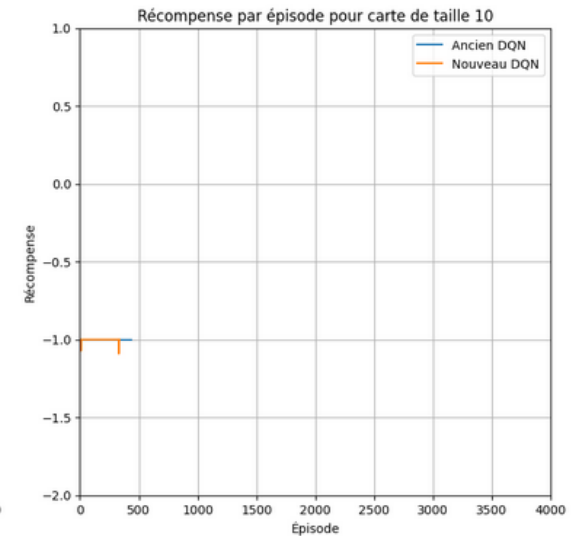
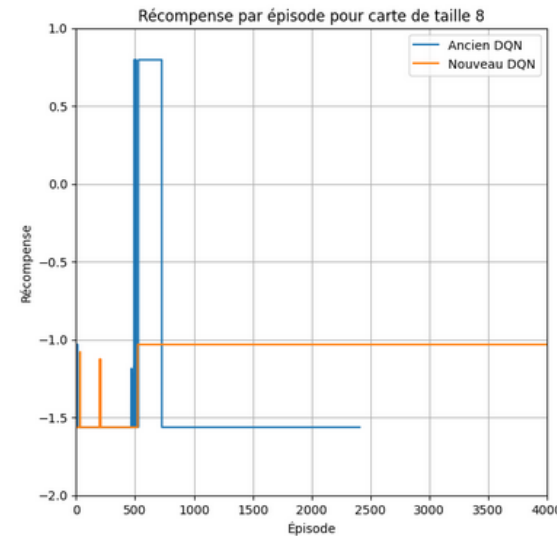
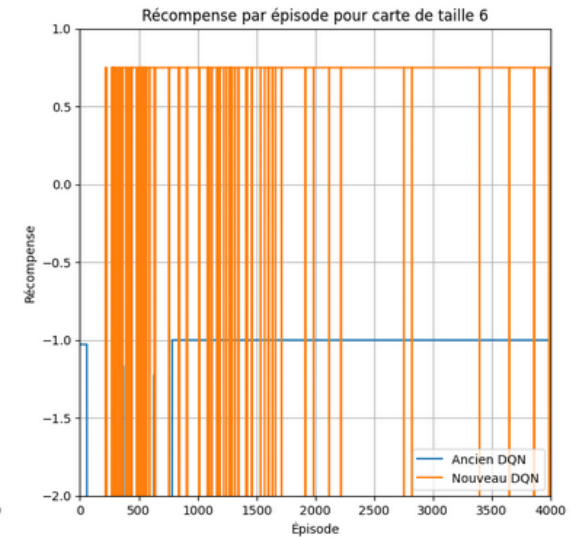
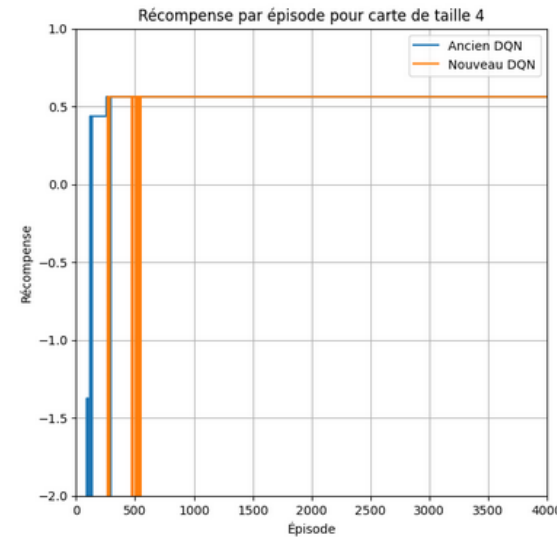
- $\alpha = 1$,
- epsilon constant très faible



Livraison - DQN

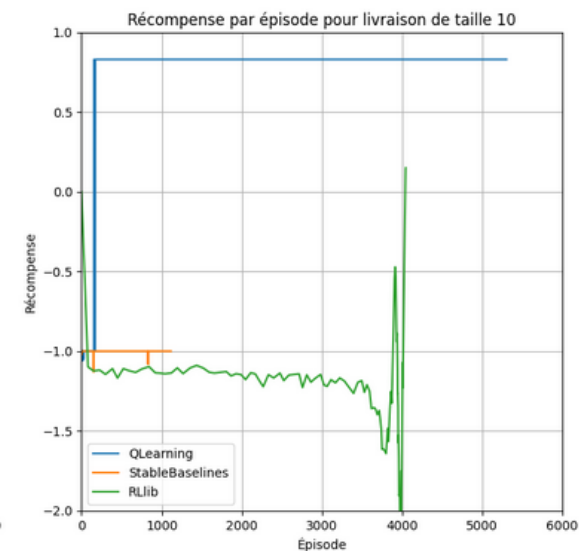
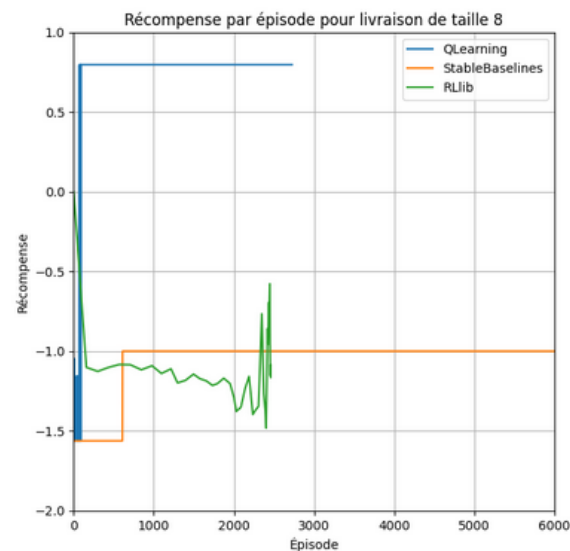
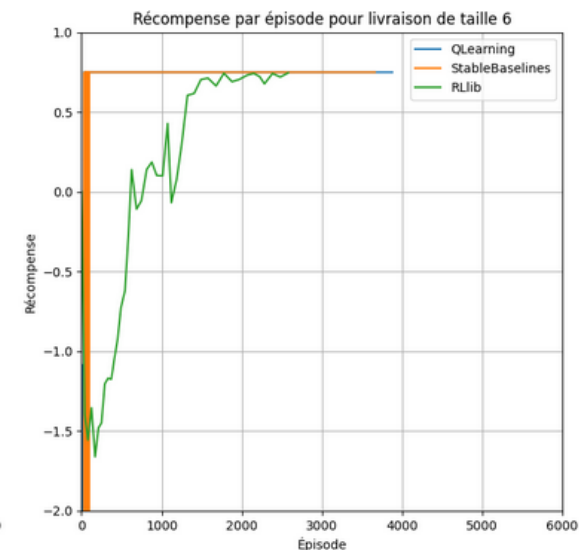
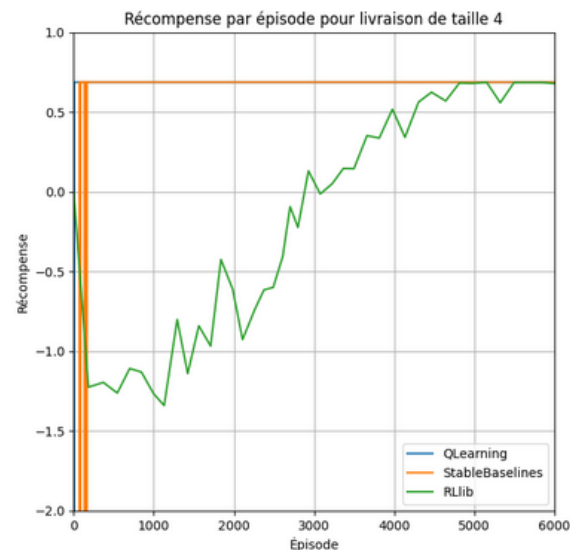
Sur plusieurs environnements de livraison :

1. L'environnement tel que présenté
2. On code la possession de la boîte séparément
3. On code la position avec deux neurones + on observe les cases adjacentes



Livraison - Résultats

Pour tailles 8 et 10, DQN apprend à aller vers un trou.



Conclusion

- QLearning ou DQN ?
- StableBaselines ou RLlib ?
- La suite ?

QLearning ou DQN ?

QLearning est plus simple à paramétrer et généralement plus rapide à entraîner.

Mais attention aux tables trop grandes

DQN peut généraliser, utile si les observations permettent de savoir quelle action effectuer.

Stablebaselines ou RLlib ?

StableBaselines :

Mieux maintenu,
plus rapide sans parallélisme,
meilleure implémentation du buffer

RLlib :

Permet le multi-agent
GridSearch intégré dans la librairie

La suite ?

Mieux expliquer
les résultats
obtenus

Quelques essais
à faire !

Découvrir de
nouveaux
algorithmes

Commencer le
multi-agent

Comment
évaluer un agent
?

Mieux visualiser
les réseaux ?